

Rapport de TP – Simulateur à événements discrets

Séance 2/3 : clients, file d’attente, serveurs, configuration et tests

Groupe 3

PINCAUD Steve, SAVIARD Matthieu et CHARONNET Henri

14 mai 2026

Résumé

Ce rapport présente le travail réalisé lors de la deuxième séance du TP consacré à la construction d’un simulateur à événements discrets. La première séance avait permis de mettre en place les briques fondamentales du simulateur : messages, événements, ordonnanceur, moteur principal et traces. Cette deuxième séance poursuit l’architecture orientée objet en introduisant les composants opérationnels du système simulé : les clients, la file d’attente, les serveurs, les distributions aléatoires et la configuration globale.

L’objectif de cette séance est de préparer la simulation d’un système de file d’attente de type M/M/1, M/M/1/K ou M/M/c/K. Pour cela, le simulateur doit être capable de créer des entités clientes, de représenter une file d’attente FIFO, de modéliser un ou plusieurs serveurs, de charger une configuration commune et de préparer la génération de temps aléatoires. Le rapport détaille les responsabilités de chaque classe, les liens entre interfaces et implémentations, la stratégie de test et l’intégration progressive dans le fichier **Engine.py**.

Table des matières

1 Introduction

La simulation à événements discrets permet d'étudier le comportement temporel d'un système en ne considérant que les instants où un changement d'état se produit. Dans le contexte du TP, le système modélisé correspond à une chaîne de traitement simplifiée composée de clients, d'une passerelle, d'une file d'attente et de serveurs.

Lors de la séance 1, les fondations du simulateur ont été mises en place :

- la classe `MessageImpl`, qui représente les messages échangés ;
- la classe `EventImpl`, qui représente les événements datés ;
- l'énumération `EventType`, qui définit les types d'événements ;
- la classe `SchedulerImpl`, qui maintient la liste chronologique des événements ;
- le fichier `Engine.py`, qui prépare le moteur de simulation ;
- le système de journalisation.

La séance 2 étend cette architecture avec les composants qui représentent l'activité réelle du système :

- la classe `ClientImpl`, chargée de modéliser les générateurs de messages ;
- la classe `QueueImpl`, chargée de représenter la file d'attente ;
- la classe `ServerImpl`, chargée de représenter les serveurs de traitement ;
- la classe `ConfigImpl`, chargée de centraliser la configuration ;
- les classes `Distribution` et `Poisson`, utilisées pour la génération aléatoire ;
- les interfaces associées : `IClient`, `IQueue`, `IServer`, `IConfig`.

Cette séance ne correspond pas encore à l'assemblage complet du simulateur final. Elle constitue cependant une étape essentielle : les objets métier nécessaires à la simulation sont maintenant identifiés, structurés et reliés par des interfaces.

2 Objectifs de la séance 2

D'après le planning du TP, la séance 2 porte sur :

- le `Main` et les méthodes de test ;
- la classe `Client` ;
- la classe `Queue` ;
- la classe `Server`.

L'objectif est donc d'enrichir le squelette du simulateur avec les entités qui interviendront directement dans la dynamique d'une file d'attente.

2.1 Objectifs fonctionnels

Les objectifs fonctionnels de cette séance sont les suivants :

1. représenter un ou plusieurs clients capables de générer des messages ;
2. représenter une file d'attente contenant les messages en attente de traitement ;
3. représenter un ou plusieurs serveurs capables de traiter des messages ;
4. préparer la génération de temps inter-arrivées et de temps de service ;
5. intégrer ces composants dans le moteur principal ;
6. mettre en place des tests séparés pour chaque classe.

2.2 Objectifs logiciels

D'un point de vue logiciel, la séance 2 poursuit plusieurs objectifs de conception :

- conserver une architecture orientée objet ;
- maintenir la séparation entre interfaces et implémentations ;
- centraliser les paramètres dans une classe de configuration ;
- utiliser le logging de manière homogène ;
- préparer l'extension vers la passerelle et les métriques de la séance 3 ;
- rendre les composants testables indépendamment.

3 Rappel du modèle simulé

3.1 Architecture générale

Le système simulé peut être représenté par la chaîne suivante :

Client(s) -> Queue -> Server(s)

Les clients génèrent des messages. Ces messages sont ensuite reçus par la passerelle. Si un serveur est disponible, le message peut être traité immédiatement. Sinon, il est placé dans une file d'attente. Lorsque le traitement se termine, un événement de départ est généré.

3.2 Événements manipulés

Les événements de base restent ceux de la séance 1 :

- SEND_MSG : émission d'un message par un client ;
- RECV_MSG : réception du message par la passerelle ;
- MSG_DEPT : fin de traitement du message par un serveur.

L'énumération `EventType` centralise ces types d'événements :

```
1 from enum import Enum
2
3 ## EventType      SEND_MSG, RECV_MSG, MSG_DEPT
4
5 class EventType(Enum):
6     SEND_MSG = 1
7     RECV_MSG = 2
8     MSG_DEPT = 3
```

Listing 1 – EventType.py

L'utilisation d'une énumération permet de limiter les erreurs liées aux chaînes de caractères libres. Elle rend également le code plus lisible, car les types d'événements sont déclarés en un seul endroit.

3.3 Variables importantes du modèle

Pour préparer les simulations de la séance 3, plusieurs variables devront être suivies :

- le temps courant de simulation ;
- le nombre de messages générés ;
- le nombre de messages reçus ;
- le nombre de messages servis ;
- le nombre de messages présents dans la file ;
- l'état des serveurs ;
- le nombre de messages rejetés dans les modèles à capacité finie ;
- les temps de séjour des messages.

La séance 2 prépare ces variables en introduisant les classes qui vont les porter ou les manipuler.

4 Organisation du projet

4.1 Arborescence logique

L'organisation du projet suit la structure suivante :

```
rcp103/
|-- tp2.log
|-- net/
    |-- lecnam/
        |-- rcp103/
```

```

|-- tp2/
|   |-- Engine.py
|   |-- ConfigImpl.py
|   |-- ConfigImplTest.py
|   |-- Distribution.py
|   |-- Poisson.py
|   |-- MessageImpl.py
|   |-- MessageImplTest.py
|   |-- EventImpl.py
|   |-- EventImplTest.py
|   |-- EventImplMockTest.py
|   |-- EventType.py
|   |-- SchedulerImpl.py
|   |-- ClientImpl.py
|   |-- ClientImplTest.py
|   |-- QueueImpl.py
|   |-- QueueImplTest.py
|   |-- ServerImpl.py
|   |-- ServerImplTest.py
|   |-- GatewayImpl.py
|   |-- IConfig.py
|   |-- IMessage.py
|   |-- IEvent.py
|   |-- IScheduler.py
|   |-- IClient.py
|   |-- IQueue.py
|   |-- IServer.py
|   |-- IGateway.py
|   |-- logging_config.cnf

```

Cette organisation permet de regrouper chaque concept dans un fichier séparé. Les fichiers préfixés par `I` correspondent aux interfaces, tandis que les fichiers suffixés par `Impl` correspondent aux implémentations concrètes.

4.2 Commandes d'exécution

Les fichiers de test peuvent être exécutés depuis la racine du projet avec la variable `PYTHONPATH` :

```

1 PYTHONPATH=. python3 net/lecnam/rcp103/tp2/ConfigImplTest.py
2 PYTHONPATH=. python3 net/lecnam/rcp103/tp2/ClientImplTest.py
3 PYTHONPATH=. python3 net/lecnam/rcp103/tp2/QueueImplTest.py
4 PYTHONPATH=. python3 net/lecnam/rcp103/tp2/ServerImplTest.py
5 PYTHONPATH=. python3 net/lecnam/rcp103/tp2/Engine.py

```

Listing 2 – Exécution des tests sous Linux ou WSL

Il est également possible d'exécuter les fichiers comme modules Python :

```

1 python3 -m net.lecnam.rcp103.tp2.ConfigImplTest
2 python3 -m net.lecnam.rcp103.tp2.ClientImplTest
3 python3 -m net.lecnam.rcp103.tp2.QueueImplTest
4 python3 -m net.lecnam.rcp103.tp2.ServerImplTest
5 python3 -m net.lecnam.rcp103.tp2.Engine

```

Listing 3 – Exécution sous forme de modules

Sous Windows, les fichiers peuvent être lancés avec :

```

1 python ConfigImplTest.py
2 python ClientImplTest.py
3 python QueueImplTest.py

```

```
4 python ServerImplTest.py
5 python Engine.py
```

Listing 4 – Exécution sous Windows PowerShell

5 Architecture orientée objet de la séance 2

5.1 Interfaces et implémentations

Le projet conserve une séparation nette entre interfaces et classes concrètes.

Les interfaces principales utilisées pendant cette séance sont :

- `IConfig`;
- `IClient`;
- `IQueue`;
- `IServer`;
- `IMessage`;
- `IEvent`;
- `IScheduler`.

Les implémentations associées sont :

- `ConfigImpl`;
- `ClientImpl`;
- `QueueImpl`;
- `ServerImpl`;
- `MessageImpl`;
- `EventImpl`;
- `SchedulerImpl`.

Cette organisation est importante, car elle permet de raisonner sur les contrats logiciels indépendamment des choix d'implémentation. Par exemple, un composant peut manipuler un objet de type `IQueue` sans dépendre directement de la classe `QueueImpl`.

5.2 Relations entre les composants

Les relations principales sont les suivantes :

- un `ClientImpl` génère ou prépare des messages;
- un `MessageImpl` contient une source, une destination et un timestamp;
- un `ServerImpl` représente une unité de traitement;
- un `QueueImpl` stocke les messages en attente;
- un `SchedulerImpl` ordonne les événements futurs;
- un `Engine` crée les clients et les serveurs;
- un `ConfigImpl` fournit les paramètres communs.

5.3 Rôle de la configuration

La configuration joue un rôle plus important dans cette séance que dans la précédente. Les fichiers `ClientImpl.py`, `QueueImpl.py`, `ServerImpl.py`, `Distribution.py` et `Poisson.py`instancient `ConfigImpl` afin de récupérer notamment le chemin du fichier de configuration des logs.

Ce choix permet de ne pas dupliquer les paramètres dans toutes les classes. Les constantes comme la graine aléatoire, le dossier de sortie ou le fichier de configuration du logging peuvent ainsi être centralisées.

6 La classe ConfigImpl

6.1 Rôle général

La classe `ConfigImpl` représente la configuration globale du simulateur. Elle est utilisée par plusieurs autres classes afin d'accéder aux paramètres communs.

D'après les fichiers de test, elle fournit au minimum :

- un dossier de sortie ;
- une graine aléatoire ;
- le chemin vers le fichier de configuration du logging ;
- une méthode d'affichage de la configuration.

6.2 Structure du fichier

Le fichier `ConfigImpl.py` commence par charger les modules nécessaires :

```
1 #!/usr/bin/python3
2
3 #####
4 ## PYTHONPATH=. /usr/bin/python3 net/lecnam/rcp103/tp2/ConfigImpl.py
5
6 import os
7 import logging
8 import logging.config
9
10 from net.lecnam.rcp103.SimulateurException import SimulateurException
11 from net.lecnam.rcp103.tp2.IConfig import IConfig
```

Listing 5 – Extrait de `ConfigImpl.py`

La classe hérite de l'interface `IConfig` :

```
1 class ConfigImpl(IConfig):
2     pass
```

Listing 6 – Déclaration de `ConfigImpl`

6.3 Test de configuration

Le fichier `ConfigImplTest.py` instancie `ConfigImpl` et vérifie l'accès aux paramètres principaux :

```
1 cfg = ConfigImpl()
2
3 out_dir = cfg.get_output_dir()
4 seed = cfg.get_seed()
5 log_path = cfg.get_log_cfg_file_path()
6 conf = cfg.print_config()
```

Listing 7 – Principe du test de `ConfigImpl`

Ce test valide que la configuration peut être chargée avant les autres composants. C'est important, car plusieurs classes dépendent de cette configuration dès leur import.

6.4 Intérêt dans le simulateur

La centralisation des paramètres présente plusieurs avantages :

- elle évite la duplication des valeurs constantes ;
- elle facilite la modification des chemins de fichiers ;
- elle permet d'utiliser une graine commune pour rendre les expériences reproductibles ;
- elle simplifie la configuration du logging ;
- elle prépare les futures campagnes de simulation avec plusieurs valeurs de paramètres.

7 Génération aléatoire : Distribution et Poisson

7.1 Rôle des distributions

Un simulateur de files d'attente probabilistes doit générer des variables aléatoires. Dans le cas des modèles M/M/1, M/M/1/K et M/M/c/K, les temps inter-arrivées et les temps de service suivent des lois exponentielles.

Le sujet rappelle que les inter-arrivées et les temps de service peuvent être générés en Python à l'aide de la méthode :

```
1 rng.exponential(scale=lam)
2 rng.exponential(scale=mu)
```

Listing 8 – Génération exponentielle avec NumPy

Dans le projet, cette logique est préparée par les classes `Distribution` et `Poisson`.

7.2 Classe Distribution

Le fichier `Distribution.py` représente une classe parente pour différentes distributions statistiques. Il initialise la configuration et le générateur pseudo-aléatoire NumPy :

```
1 cfg = ConfigImpl()
2 log_path = cfg.get_log_cfg_file_path()
3
4 seed = cfg.get_seed()
5 rng = np.random.default_rng(seed=seed)
6
7 class Distribution:
8     "Classe de base repr sentant une distribution statistique."
```

Listing 9 – Extrait conceptuel de `Distribution.py`

L'utilisation d'une graine aléatoire est importante pour la reproductibilité. Deux exécutions avec la même graine doivent produire les mêmes suites de nombres pseudo-aléatoires, ce qui facilite les tests et la comparaison des résultats.

7.3 Classe Poisson

La classe `Poisson` hérite de `Distribution` :

```
1 from net.lecnam.rcp103.tp2.Distribution import Distribution
2
3 class Poisson(Distribution):
4     "Distribution de Poisson."
```

Listing 10 – Extrait conceptuel de `Poisson.py`

Même si les modèles M/M/1 utilisent principalement des temps exponentiels, la présence d'une classe `Poisson` permet de préparer une abstraction plus générale des distributions. Elle peut servir à modéliser des comptages ou à structurer les générateurs aléatoires du simulateur.

7.4 Lien avec les files d'attente

Dans une file M/M/1 :

- les arrivées suivent un processus de Poisson ;
- les temps inter-arrivées suivent une loi exponentielle ;
- les temps de service suivent aussi une loi exponentielle ;
- le système possède un serveur unique ;
- la file est infinie dans le cas M/M/1.

La mise en place des classes de distribution pendant la séance 2 prépare donc directement les simulations quantitatives prévues en séance 3.

8 La classe ClientImpl

8.1 Rôle du client

La classe `ClientImpl` représente un client ou un générateur de messages. Dans le modèle du TP, un client est une entité capable de produire des messages à destination de la passerelle ou d'un serveur.

Son rôle est donc de préparer :

- la génération de messages;
- l'identification de la source;
- l'association d'une destination;
- la génération d'événements de type `SEND_MSG`;
- la future programmation des arrivées dans le scheduler.

8.2 Dépendances de ClientImpl

Le fichier `ClientImpl.py` importe plusieurs composants du simulateur :

```
1 from net.lecnam.rcp103.tp2.IEvent import IEvent
2 from net.lecnam.rcp103.tp2.IMessage import IMessage
3 from net.lecnam.rcp103.tp2.EventType import EventType
4 from net.lecnam.rcp103.tp2.IClient import IClient
5 from net.lecnam.rcp103.tp2.ConfigImpl import ConfigImpl
6 from net.lecnam.rcp103.SimulateurException import SimulateurException
7 from net.lecnam.rcp103.tp2.Poisson import Poisson
8 from net.lecnam.rcp103.tp2.Poisson import Distribution
9 from net.lecnam.rcp103.tp2.QueueImpl import QueueImpl
10 from net.lecnam.rcp103.tp2.IServer import IServer
```

Listing 11 – Imports principaux de `ClientImpl.py`

Ces imports montrent que le client est placé au croisement de plusieurs responsabilités :

- il manipule des messages;
- il peut être associé à un serveur;
- il s'intègre à la file d'attente;
- il dépend de la configuration;
- il prépare l'utilisation de distributions aléatoires.

8.3 Interface IClient

L'interface `IClient` hérite de `ABC`. Elle importe `IMessage` et `IServer`, ce qui indique qu'un client est lié à la création ou à l'envoi de messages vers une entité de traitement :

```
1 from abc import ABC, abstractmethod
2
3 from net.lecnam.rcp103.tp2.IMessage import IMessage
4 from net.lecnam.rcp103.tp2.IServer import IServer
5
6 class IClient(ABC):
7     pass
```

Listing 12 – Structure de `IClient.py`

8.4 Configuration du logging

Comme les autres composants, `ClientImpl` récupère la configuration via `ConfigImpl` :

```
1 cfg = ConfigImpl()
2 log_path = cfg.get_log_cfg_file_path()
3
4 config_path = os.path.join(os.path.dirname(__file__), log_path)
5 logging.config.fileConfig(config_path)
```



```

6     config_path,
7     defaults=None,
8     disable_existing_loggers=False,
9     encoding=None
10 )
11
12 logger = logging.getLogger(__name__)

```

Listing 13 – Configuration du logging dans ClientImpl.py

Cela garantit que les traces produites par les clients utilisent le même format que celles du reste du simulateur.

8.5 Comportement attendu

Dans la version complète du simulateur, un client devra être capable de :

1. générer un identifiant de message ;
2. créer une instance de `MessageImpl` ;
3. affecter le timestamp au temps courant de simulation ;
4. créer un événement `SEND_MSG` ;
5. insérer cet événement dans le scheduler ;
6. planifier éventuellement le prochain envoi.

Une structure conceptuelle possible est :

```

1 def generate_message(self, current_time):
2     message = MessageImpl(
3         message_id=self.next_message_id(),
4         source=self.client_id,
5         destination=self.destination,
6         timestamp=current_time
7     )
8     return message

```

Listing 14 – Principe conceptuel de génération d'un message

9 La classe QueueImpl

9.1 Rôle de la file d'attente

La classe `QueueImpl` représente la file d'attente du système. Elle stocke les messages qui ne peuvent pas être traités immédiatement parce que tous les serveurs sont occupés.

Dans une file d'attente de type FIFO, le premier message arrivé est le premier message servi. Ce comportement correspond à la politique *First In, First Out*.

9.2 Dépendances de QueueImpl

Le fichier `QueueImpl.py` importe notamment :

```

1 from net.lecnam.rcp103.tp2.IQueue import IQueue
2 from net.lecnam.rcp103.tp2.ConfigImpl import ConfigImpl
3 from net.lecnam.rcp103.SimulateurException import SimulateurException
4
5 import queue
6 import numpy as np
7
8 from net.lecnam.rcp103.tp2.IMessage import IMessage

```

Listing 15 – Imports principaux de QueueImpl.py

Ces imports indiquent que `QueueImpl` :

- implémente l'interface `IQueue` ;

- manipule des objets respectant l'interface `IMessage` ;
- peut utiliser le module standard `queue` de Python ;
- dépend de la configuration commune ;
- est préparée pour les traitements numériques ou statistiques via `numpy`.

9.3 Interface IQueue

L'interface `IQueue` est définie comme une classe abstraite :

```

1 from abc import ABC, abstractmethod
2
3 from net.lecnam.rcp103.tp2 import IMessage
4
5 class IQueue(ABC):
6     pass

```

Listing 16 – Structure de `IQueue.py`

Elle représente le contrat minimal que devra respecter toute implémentation de file d'attente.

9.4 Opérations attendues

Une file d'attente du simulateur devra fournir les opérations suivantes :

- ajouter un message dans la file ;
- retirer le prochain message ;
- tester si la file est vide ;
- retourner la taille actuelle ;
- vérifier si la file est pleine dans les modèles à capacité finie ;
- afficher le contenu ou l'état de la file.

Une implémentation conceptuelle pourrait suivre le schéma suivant :

```

1 class QueueImpl(IQueue):
2
3     def __init__(self, capacity=None):
4         self.messages = []
5         self.capacity = capacity
6
7     def enqueue(self, message):
8         if self.capacity is not None and len(self.messages) >= self.capacity:
9             return False
10        self.messages.append(message)
11        return True
12
13    def dequeue(self):
14        if not self.messages:
15            return None
16        return self.messages.pop(0)
17
18    def is_empty(self):
19        return len(self.messages) == 0
20
21    def size(self):
22        return len(self.messages)

```

Listing 17 – Principe conceptuel d'une file FIFO

9.5 Lien avec les modèles M/M/1/K

Dans le modèle M/M/1, la file est supposée infinie. Dans les modèles M/M/1/K ou M/M/c/K, la capacité totale du système est limitée à K . Cela signifie que la file doit être capable de refuser un message lorsque la capacité maximale est atteinte.

Pour un système avec c serveurs et une capacité totale K , la capacité effective de la file peut être vue comme :

$$K_{\text{file}} = K - c$$

Ainsi, pour un modèle M/M/3/8, la capacité totale est $K = 8$ et le nombre de serveurs est $c = 3$. La file peut donc contenir au plus :

$$K_{\text{file}} = 8 - 3 = 5$$

messages en attente, si les trois serveurs sont occupés.

10 La classe `ServerImpl`

10.1 Rôle du serveur

La classe `ServerImpl` représente un serveur de traitement. Dans le modèle étudié, un serveur reçoit un message, le traite pendant un temps aléatoire, puis déclenche un événement de départ `MSG_DEPT`.

Un serveur doit notamment posséder :

- un identifiant ;
- un état libre ou occupé ;
- éventuellement le message en cours de traitement ;
- une méthode permettant de démarrer un service ;
- une méthode permettant de terminer un service ;
- une méthode d’affichage.

10.2 Dépendances de `ServerImpl`

Le fichier `ServerImpl.py` importe les composants suivants :

```
1 from net.lecnam.rcp103.tp2.IEvent import IEvent
2 from net.lecnam.rcp103.tp2.IMessage import IMessage
3 from net.lecnam.rcp103.tp2.EventType import EventType
4 from net.lecnam.rcp103.tp2.IClient import IClient
5 from net.lecnam.rcp103.tp2.IQueue import IQueue
6 from net.lecnam.rcp103.tp2.IServer import IServer
7 from net.lecnam.rcp103.tp2.ConfigImpl import ConfigImpl
8 from net.lecnam.rcp103.SimulateurException import SimulateurException
```

Listing 18 – Imports principaux de `ServerImpl.py`

Ces dépendances montrent que le serveur est lié :

- aux messages à traiter ;
- aux événements de fin de traitement ;
- à la file d’attente ;
- à la configuration ;
- aux interfaces générales du simulateur.

10.3 Interface `IServer`

L’interface `IServer` est déclarée comme une classe abstraite :

```
1 from abc import ABC, abstractmethod
2
3 from net.lecnam.rcp103.tp2 import IQueue
4 from net.lecnam.rcp103.tp2 import IMessage
5
6 class IServer(ABC):
7     pass
```

Listing 19 – Structure de `IServer.py`

Cette interface prépare le contrat des serveurs. Dans une version complète, elle pourra déclarer des méthodes comme :

```
— is_busy();  
— start_service(message);  
— end_service();  
— get_current_message();  
— print_server().
```

10.4 Gestion du logging

Le fichier `ServerImpl.py` contient une gestion particulière de la configuration du logging :

```
1 try:  
2     cfg = ConfigImpl()  
3     log_path = cfg.get_log_cfg_file_path()  
4  
5 except Exception as e:  
6     # Fallback to basic logging if config file fails  
7     logging.basicConfig(  
8         level=logging.DEBUG,  
9         format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'  
10    )  
11    logger = logging.getLogger(__name__)  
12    logger.error(f"Failed to load logging config from : ")
```

Listing 20 – Principe de chargement de configuration dans `ServerImpl.py`

Ce mécanisme permet de conserver des traces même si le chargement du fichier de configuration échoue. Cela est utile pendant les phases de développement et de test.

10.5 Comportement attendu

Dans le simulateur complet, le serveur devra fonctionner selon la logique suivante :

1. lorsqu'un message arrive, vérifier si le serveur est libre ;
2. si le serveur est libre, démarrer immédiatement le service ;
3. générer un temps de service aléatoire ;
4. programmer un événement `MSG_DEPT` au temps courant plus le temps de service ;
5. marquer le serveur comme occupé ;
6. à la fin du service, libérer le serveur ;
7. si la file n'est pas vide, extraire le prochain message et démarrer son service.

Le comportement conceptuel peut être représenté ainsi :

```
1 def start_service(self, message, current_time, service_time):  
2     self.busy = True  
3     self.current_message = message  
4     departure_time = current_time + service_time  
5     return departure_time  
6  
7 def end_service(self):  
8     completed_message = self.current_message  
9     self.current_message = None  
10    self.busy = False  
11    return completed_message
```

Listing 21 – Principe conceptuel de traitement par un serveur

11 Évolution de MessageImpl

11.1 Rôle du message

La classe `MessageImpl` reste une brique centrale du simulateur. Un message transporte les informations nécessaires pour suivre son parcours dans le système :

- son identifiant ;
- sa source ;
- sa destination ;
- son timestamp de création.

11.2 Destination typée par IServer

Dans la version fournie pour la séance 2, la destination est typée comme un `IServer` :

```
1 class MessageImpl(IMessage):
2     """ Message envoy vers la passerelle """
3
4     message_id: int
5     source: str
6     destination: IServer
7     timestamp: float
```

Listing 22 – Attributs de MessageImpl

Ce choix rapproche le message de la logique de traitement : le message peut être associé directement à un serveur destinataire ou à une entité capable de fournir une méthode d’affichage de type `print_server()`.

11.3 Méthode print_message

La méthode d’affichage utilise la destination :

```
1 def print_message(self):
2     return(f"[Message] ID={self._message_id} "
3           f"src={self._source} "
4           f"dst={self._destination.print_server()} "
5           f"timestamp={self._timestamp:.4f}")
```

Listing 23 – Affichage d’un message

Cette méthode permet de produire une trace textuelle complète du message. Elle sera utile pour le debugging et pour la génération des traces d’événements.

12 Évolution du moteur Engine.py

12.1 Rôle du moteur

Le moteur `Engine` est le point d’entrée du simulateur. Lors de la séance 2, il commence à intégrer les nouveaux composants : clients, serveurs, file d’attente, distributions et configuration.

12.2 Imports du moteur

La version fournie du fichier `Engine.py` importe notamment :

```
1 from net.lecnam.rcp103.tp2.MessageImpl import MessageImpl
2 from net.lecnam.rcp103.tp2.EventImpl import EventImpl
3 from net.lecnam.rcp103.tp2.SchedulerImpl import SchedulerImpl
4 from net.lecnam.rcp103.tp2.IScheduler import IScheduler
5 from net.lecnam.rcp103.tp2.IServer import IServer
6 from net.lecnam.rcp103.tp2.IClient import IClient
7 from net.lecnam.rcp103.tp2.IQueue import IQueue
8 from net.lecnam.rcp103.tp2.ServerImpl import ServerImpl
```

```

9 from net.lecnam.rcp103.tp2.ClientImpl import ClientImpl
10 from net.lecnam.rcp103.tp2.QueueImpl import QueueImpl
11 from net.lecnam.rcp103.tp2.Poisson import Poisson
12 from net.lecnam.rcp103.tp2.Poisson import Distribution
13 from net.lecnam.rcp103.tp2.ConfigImpl import ConfigImpl

```

Listing 24 – Imports principaux de Engine.py

Cela montre que le moteur devient progressivement l'orchestrateur global du simulateur.

12.3 Création des clients et des serveurs

Dans le bloc principal, le moteur crée plusieurs serveurs et plusieurs clients :

```

1 if __name__ == "__main__":
2     logger.debug("+++ Engine : Main START ...")
3     engine = Engine()
4     engine.create_servers(2)
5     engine.create_clients(4)

```

Listing 25 – Bloc principal de Engine.py

Cette structure prépare les simulations multi-clients et multi-serveurs. Elle sera particulièrement utile pour le modèle M/M/c/K, où plusieurs serveurs partagent une même file d'attente.

12.4 Méthodes attendues dans Engine

À terme, le moteur devra contenir des méthodes comme :

- create_clients(n) ;
- create_servers(n) ;
- create_queue(capacity) ;
- run() ;
- generate_trace(event) ;
- test_client() ;
- test_queue() ;
- test_server() .

Le moteur devra également contenir la boucle principale de simulation :

```

1 while scheduler.has_events():
2     event = scheduler.get_event()
3     current_time = scheduler.get_current_time()
4
5     if event.get_event_type() == "SEND_MSG":
6         handle_send_msg(event)
7
8     elif event.get_event_type() == "RCV_MSG":
9         handle_rcv_msg(event)
10
11    elif event.get_event_type() == "MSG_DEPT":
12        handle_msg_dept(event)

```

Listing 26 – Boucle principale conceptuelle

13 Tests réalisés

13.1 Méthodologie générale

La méthodologie de test suivie dans le TP est incrémentale. Chaque classe possède un fichier de test dédié. Cela permet de valider les composants un par un avant de les assembler dans le moteur complet.

Les tests de la séance 2 concernent principalement :

- ConfigImplTest.py;
- ClientImplTest.py;
- QueueImplTest.py;
- ServerImplTest.py;
- MessageImplTest.py.

13.2 Test de ConfigImpl

Le test de configuration vérifie que l'objet `ConfigImpl` peut être instancié et que ses principales méthodes peuvent être appelées :

```
1 cfg = ConfigImpl()
2 out_dir = cfg.get_output_dir()
3 seed = cfg.get_seed()
4 log_path = cfg.get_log_cfg_file_path()
5 conf = cfg.print_config()
```

Listing 27 – Extrait de ConfigImplTest.py

Ce test est important, car les autres composants dépendent de la configuration pour charger le logging.

13.3 Test de ClientImpl

Le fichier `ClientImplTest.py` importe les composants nécessaires au test du client :

```
1 from net.lecnam.rcp103.tp2.ConfigImpl import ConfigImpl
2 from net.lecnam.rcp103.tp2.ClientImpl import ClientImpl
3 from net.lecnam.rcp103.tp2.ServerImpl import ServerImpl
4 from net.lecnam.rcp103.tp2.QueueImpl import QueueImpl
5
6 cfg = ConfigImpl()
7 log_path = cfg.get_log_cfg_file_path()
```

Listing 28 – Extrait de ClientImplTest.py

Le test vérifie d'abord que la configuration peut être chargée avant d'instancier ou de manipuler les objets liés au client.

13.4 Test de QueueImpl

Le fichier `QueueImplTest.py` importe `QueueImpl`, la configuration et les modules nécessaires :

```
1 from net.lecnam.rcp103.tp2.ConfigImpl import ConfigImpl
2 from net.lecnam.rcp103.tp2.QueueImpl import QueueImpl
3
4 cfg = ConfigImpl()
5 log_path = cfg.get_log_cfg_file_path()
```

Listing 29 – Extrait de QueueImplTest.py

Dans une version complète, ce test devra vérifier :

- l'ajout d'un message;
- le retrait d'un message;
- le respect de l'ordre FIFO;
- la taille de la file;
- le comportement lorsque la file est pleine.

Un test conceptuel utile serait :

```
1 queue = QueueImpl()
2
3 queue.enqueue(msg1)
4 queue.enqueue(msg2)
```

```

5 queue.enqueue(msg3)
6
7 assert queue.dequeue() == msg1
8 assert queue.dequeue() == msg2
9 assert queue.dequeue() == msg3
10 assert queue.is_empty()

```

Listing 30 – Test conceptuel du comportement FIFO

13.5 Test de ServerImpl

Le fichier `ServerImplTest.py` importe les classes nécessaires au test du serveur :

```

1 from net.lecnam.rcp103.tp2.ConfigImpl import ConfigImpl
2 from net.lecnam.rcp103.tp2.ServerImpl import ServerImpl
3 from net.lecnam.rcp103.tp2.MessageImpl import MessageImpl
4 from net.lecnam.rcp103.tp2.QueueImpl import QueueImpl
5 from net.lecnam.rcp103.tp2.Poisson import Poisson
6 from net.lecnam.rcp103.tp2.Poisson import Distribution
7
8 cfg = ConfigImpl()
9 log_path = cfg.get_log_cfg_file_path()

```

Listing 31 – Extrait de `ServerImplTest.py`

Ce test prépare la validation du comportement serveur : traitement d'un message, changement d'état, utilisation d'une file et génération d'un temps de service.

Un test conceptuel possible serait :

```

1 server = ServerImpl(server_id=1)
2 message = MessageImpl(1, "client_1", server, 0.0)
3
4 assert not server.is_busy()
5
6 server.start_service(message)
7
8 assert server.is_busy()
9 assert server.get_current_message() == message
10
11 completed = server.end_service()
12
13 assert completed == message
14 assert not server.is_busy()

```

Listing 32 – Test conceptuel d'un serveur

13.6 Test de MessageImpl

Le test de `MessageImpl` crée un message, récupère sa source et sa destination, puis affiche son contenu :

```

1 impl = MessageImpl(1, "Alice", "Bob", 1.21)
2
3 src = impl.get_source()
4 dst = impl.get_destination()
5
6 res = impl.print_message()

```

Listing 33 – Extrait de `MessageImplTest.py`

Ce test permet de vérifier que les accesseurs et la méthode d'affichage du message fonctionnent correctement.

14 Journalisation

14.1 Rôle du logging

Le logging reste un élément important du simulateur. Il permet :

- de vérifier le chargement des classes ;
- de suivre les tests ;
- de détecter les erreurs d'import ou de configuration ;
- de produire une trace utile au débogage ;
- de préparer la génération de traces exploitables pour les métriques.

14.2 Chargement homogène de la configuration

La plupart des classes de la séance 2 suivent la même logique :

1. instancier `ConfigImpl` ;
2. récupérer le chemin du fichier de configuration du logging ;
3. charger ce fichier avec `logging.config.fileConfig` ;
4. créer un logger local avec `logging.getLogger(__name__)`.

Le schéma général est :

```

1  cfg = ConfigImpl()
2  log_path = cfg.get_log_cfg_file_path()
3
4  config_path = os.path.join(os.path.dirname(__file__), log_path)
5
6  logging.config.fileConfig(
7      config_path,
8      defaults=None,
9      disable_existing_loggers=False,
10     encoding=None
11 )
12
13 logger = logging.getLogger(__name__)

```

Listing 34 – Schéma général de configuration du logging

14.3 Exemple de mécanisme de secours

Le serveur ajoute une logique de secours si la configuration échoue :

```

1  except Exception as e:
2      logging.basicConfig(
3          level=logging.DEBUG,
4          format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
5      )
6      logger = logging.getLogger(__name__)
7      logger.error(f"Failed to load logging config from : ")

```

Listing 35 – Fallback logging dans `ServerImpl.py`

Ce mécanisme permet de conserver une visibilité sur les erreurs même lorsque le fichier de configuration n'est pas accessible.

15 Validation expérimentale par les traces d'exécution

15.1 Objectif de la validation

Afin de vérifier que le code développé pendant la séance 2 fonctionne correctement, nous avons ajouté une étape de validation expérimentale fondée sur l'observation des traces d'exécution. Cette étape est importante, car elle permet de montrer que le simulateur ne se limite pas à une structure de classes, mais qu'il exécute effectivement une partie des calculs attendus.

En particulier, le fichier `Engine.py` contient une méthode nommée `calcul_MM1_rate`. Cette méthode calcule et affiche les valeurs théoriques associées à une file M/M/1 pour différentes valeurs du taux d'arrivée λ . Les grandeurs affichées sont :

- λ , le taux d'arrivée des messages ;
- ρ , le taux d'utilisation du serveur ;
- L , le nombre moyen de messages dans le système ;
- W , le temps moyen passé dans le système.

Les valeurs utilisées correspondent au cas présenté dans le sujet du TP :

$$\lambda \in \{4, 6, 8, 12\}$$

avec un taux de service :

$$\mu = 8$$

15.2 Rappel des formules théoriques M/M/1

Pour une file M/M/1 stable, le taux d'utilisation est donné par :

$$\rho = \frac{\lambda}{\mu}$$

Le nombre moyen de messages dans le système est :

$$L = \frac{\rho}{1 - \rho}$$

Le temps moyen dans le système est :

$$W = \frac{1}{\mu - \lambda}$$

Ces formules ne sont valides que lorsque la condition de stabilité est vérifiée :

$$\rho < 1 \iff \lambda < \mu$$

Lorsque $\lambda = \mu$, le système atteint la limite de stabilité. Dans ce cas, le dénominateur devient nul dans les formules de L et W . Le programme affiche alors la valeur `INFINI`, conformément au tableau théorique présenté dans le support de cours.

Lorsque $\lambda > \mu$, le système est instable : le taux d'arrivée est supérieur à la capacité moyenne de service. Dans le cadre de ce TP, l'affichage de valeurs divergentes permet de mettre en évidence cette perte de stabilité.

15.3 Méthode utilisée dans `Engine.py`

La méthode suivante a été ajoutée dans la classe `Engine`. Elle affiche un tableau contenant les valeurs de λ , ρ , L et W .

```

1 def calcul_MM1_rate(self):
2     logger.debug("+++ Engine : START calcul_MM1_rate ...")
3     print("\n--- TRACE ---")
4     print(f"{'Lambda'} \t \t {'Rho'} \t \t {'L'} \t \t {'W'}")
5
6     # Iterate lambda_arrival_rate and calculate L, W, rho
7     # for each lambda and print the results
8     for lam in self.lambda_arrival_rate:
9         rho = lam / self.service_rate
10        try:
11            # self.lambda_arrival_rate = [4, 6, 8, 12]
12            L = rho / (1 - rho)
13            W = 1 / (self.service_rate - lam)
14            print(f"{lam:<8.3f} \t {rho:<8.3f} \t {L:<8.3f} \t {W:<8.3f}")
15        except ZeroDivisionError as e:

```

```

16         print(f"{lam:<8.3f} \t {rho:<8.3f} \t {'INFINI'} \t {'INFINI'}")
17         logger.error("Division by zero error in calcul_MM1_rate")
18
19     print("-" * 45)
20     logger.debug("+++ Engine : END calcul_MM1_rate ...")

```

Listing 36 – Méthode calcul_MM1.rate dans Engine.py

Cette méthode commence par afficher un en-tête de trace, puis parcourt la liste des valeurs de λ . Pour chaque valeur, elle calcule le taux d'utilisation ρ , le nombre moyen de messages L et le temps moyen W .

Le bloc `try/except` permet de gérer le cas particulier où $\lambda = \mu$. Dans ce cas, les formules contiennent une division par zéro. L'exception `ZeroDivisionError` est interceptée afin d'afficher explicitement `INFINI` dans le tableau.

15.4 Interprétation attendue des résultats

Avec les paramètres :

$$\lambda \in \{4, 6, 8, 12\} \quad \text{et} \quad \mu = 8$$

les résultats théoriques attendus sont les suivants :

λ	$\rho = \lambda/\mu$	L	W
4	0.5	1.0	0.250
6	0.75	3.0	0.500
8	1.0	INFINI	INFINI
12	1.5	système instable	système instable

Pour $\lambda = 4$, le serveur est utilisé à 50%. Le système est stable et le nombre moyen de messages dans le système vaut $L = 1$.

Pour $\lambda = 6$, le serveur est utilisé à 75%. Le système reste stable, mais la charge augmente : le nombre moyen de messages passe à $L = 3$, et le temps moyen dans le système passe à $W = 0.5$ seconde.

Pour $\lambda = 8$, on obtient $\rho = 1$. Le serveur est utilisé à 100%, ce qui correspond à la limite de stabilité. Les formules théoriques divergent, d'où l'affichage `INFINI`.

Pour $\lambda = 12$, on obtient $\rho = 1.5$. Le système reçoit en moyenne plus de messages qu'il ne peut en traiter. Le modèle M/M/1 à capacité infinie devient donc instable.

15.5 Captures d'écran des logs et de l'affichage console

Les captures suivantes montrent l'exécution réelle du programme. Elles permettent de vérifier que la méthode `calcul_MM1_rate` est bien appelée, que les logs sont produits et que le tableau de résultats est affiché.

```

pinpin@westpoint:/mnt/c/github/rcp103$ python3 -m net.lecnam.rcp103.tp2.Engine
2026-05-13 21:33:17 INFO __main__: +++ Engine : Server created:[SERVER] ID=1 | Mu=8
2026-05-13 21:33:17 INFO __main__: +++ Engine : Server created:[SERVER] ID=2 | Mu=8
2026-05-13 21:33:17 INFO __main__: +++ Engine : n=4
2026-05-13 21:33:17 INFO __main__: +++ Engine : Clients créés :ClientID=1 | ArrivalRate=4 | Queue=Empty | Destination=[SERVER] ID=1 | Mu=8
2026-05-13 21:33:17 INFO __main__: +++ Engine : Clients créés :ClientID=2 | ArrivalRate=6 | Queue=Empty | Destination=[SERVER] ID=1 | Mu=8
2026-05-13 21:33:17 INFO __main__: +++ Engine : Clients créés :ClientID=3 | ArrivalRate=8 | Queue=Empty | Destination=[SERVER] ID=1 | Mu=8
2026-05-13 21:33:17 INFO __main__: +++ Engine : Clients créés :ClientID=4 | ArrivalRate=12 | Queue=Empty | Destination=[SERVER] ID=1 | Mu=8

--- TRACE ---
Lambda      Rho      L      W
4.000      0.500    1.000    0.250
6.000      0.750    3.000    0.500
8.000      1.000    INFINI    INFINI
2026-05-13 21:33:17 ERROR __main__: Division by zero error in calcul_MM1_rate
12.000      1.500    -3.000    -0.250
-----
2026-05-13 21:33:17 INFO net.lecnam.rcp103.tp2.Poisson: +++ Poisson __init__ seed=3, lam=4

```

FIGURE 1 – Affichage du tableau théorique M/M/1 avec les valeurs de λ , ρ , L et W

La figure ?? constitue la validation principale. Elle montre que le programme affiche bien le tableau des valeurs théoriques de la file M/M/1. On vérifie notamment la présence de la ligne correspondant à $\lambda = 8$, pour laquelle les colonnes L et W affichent INFINI. Cet affichage confirme que le cas de division par zéro est bien pris en compte dans le code.

```
rcp103 > tp2.log
1 2026-05-13 20:30:58 INFO __main__ : +++ Engine : Server created:[SERVER] ID=1 | Mu=8
2 2026-05-13 20:30:58 INFO __main__ : +++ Engine : Server created:[SERVER] ID=2 | Mu=8
3 2026-05-13 20:30:58 INFO __main__ : +++ Engine : n=4
4 2026-05-13 20:30:58 INFO __main__ : +++ Engine : Clients créés :ClientID=1 | ArrivalRate=4 | Queue=Empty | Destination=[SERVER] ID=1 | Mu=8
5 2026-05-13 20:30:58 INFO __main__ : +++ Engine : Clients créés :ClientID=2 | ArrivalRate=6 | Queue=Empty | Destination=[SERVER] ID=1 | Mu=8
6 2026-05-13 20:30:58 INFO __main__ : +++ Engine : Clients créés :ClientID=3 | ArrivalRate=8 | Queue=Empty | Destination=[SERVER] ID=1 | Mu=8
7 2026-05-13 20:30:58 INFO __main__ : +++ Engine : Clients créés :ClientID=4 | ArrivalRate=12 | Queue=Empty | Destination=[SERVER] ID=1 | Mu=8
8 2026-05-13 20:30:58 ERROR __main__ : Division by zero error in calcul_MM1_rate
9 2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.Poisson: +++ Poisson __init__ seed=3, lam=4
10 2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.Poisson: +++ Poisson __init__ seed=3, lam=6
11 2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.Poisson: +++ Poisson __init__ seed=3, lam=8
12 2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.Poisson: +++ Poisson __init__ seed=3, lam=12
13 2026-05-13 20:30:58 INFO __main__ : +++ Engine run_simulationMM1: server about to start listening:[SERVER] ID=1 | Mu=8
14 2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.QueueImpl: +++ QueueImpl : Dequeued message with id=1
15 2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.ServerImpl: +++ ServerImpl : Message dequeued : [Message] ID=1 | src=1 | dst=[SERVER] ID=1 | Mu=8
16 2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.QueueImpl: +++ QueueImpl : Dequeued message with id=2
17 2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.ServerImpl: +++ ServerImpl : Message dequeued : [Message] ID=2 | src=1 | dst=[SERVER] ID=1 | Mu=8
18 2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.QueueImpl: +++ QueueImpl : Dequeued message with id=3
19 2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.ServerImpl: +++ ServerImpl : Message dequeued : [Message] ID=3 | src=1 | dst=[SERVER] ID=1 | Mu=8
20 2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.QueueImpl: +++ QueueImpl : Dequeued message with id=4
21 2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.ServerImpl: +++ ServerImpl : Message dequeued : [Message] ID=4 | src=1 | dst=[SERVER] ID=1 | Mu=8
22 2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.QueueImpl: +++ QueueImpl : Dequeued message with id=5
23 2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.ServerImpl: +++ ServerImpl : Message dequeued : [Message] ID=5 | src=1 | dst=[SERVER] ID=1 | Mu=8
24 2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.QueueImpl: +++ QueueImpl : Dequeued message with id=6
25 2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.ServerImpl: +++ ServerImpl : Message dequeued : [Message] ID=6 | src=2 | dst=[SERVER] ID=1 | Mu=8
26 2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.QueueImpl: +++ QueueImpl : Dequeued message with id=7
27 2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.ServerImpl: +++ ServerImpl : Message dequeued : [Message] ID=7 | src=2 | dst=[SERVER] ID=1 | Mu=8
```

FIGURE 2 – Première capture d’écran des logs de la séance 2

```
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.QueueImpl: +++ QueueImpl : Dequeued message with id=8
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.ServerImpl: +++ ServerImpl : Message dequeued : [Message] ID=8 | src=2 | dst=[SERVER] ID=1 | Mu=8
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.QueueImpl: +++ QueueImpl : Dequeued message with id=9
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.ServerImpl: +++ ServerImpl : Message dequeued : [Message] ID=9 | src=2 | dst=[SERVER] ID=1 | Mu=8
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.QueueImpl: +++ QueueImpl : Dequeued message with id=10
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.ServerImpl: +++ ServerImpl : Message dequeued : [Message] ID=10 | src=2 | dst=[SERVER] ID=1 | Mu=8
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.QueueImpl: +++ QueueImpl : Dequeued message with id=11
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.ServerImpl: +++ ServerImpl : Message dequeued : [Message] ID=11 | src=3 | dst=[SERVER] ID=1 | Mu=8
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.QueueImpl: +++ QueueImpl : Dequeued message with id=12
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.ServerImpl: +++ ServerImpl : Message dequeued : [Message] ID=12 | src=3 | dst=[SERVER] ID=1 | Mu=8
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.QueueImpl: +++ QueueImpl : Dequeued message with id=13
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.ServerImpl: +++ ServerImpl : Message dequeued : [Message] ID=13 | src=3 | dst=[SERVER] ID=1 | Mu=8
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.QueueImpl: +++ QueueImpl : Dequeued message with id=14
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.ServerImpl: +++ ServerImpl : Message dequeued : [Message] ID=14 | src=3 | dst=[SERVER] ID=1 | Mu=8
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.QueueImpl: +++ QueueImpl : Dequeued message with id=15
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.ServerImpl: +++ ServerImpl : Message dequeued : [Message] ID=15 | src=3 | dst=[SERVER] ID=1 | Mu=8
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.QueueImpl: +++ QueueImpl : Dequeued message with id=16
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.ServerImpl: +++ ServerImpl : Message dequeued : [Message] ID=16 | src=4 | dst=[SERVER] ID=1 | Mu=8
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.QueueImpl: +++ QueueImpl : Dequeued message with id=17
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.ServerImpl: +++ ServerImpl : Message dequeued : [Message] ID=17 | src=4 | dst=[SERVER] ID=1 | Mu=8
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.QueueImpl: +++ QueueImpl : Dequeued message with id=18
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.ServerImpl: +++ ServerImpl : Message dequeued : [Message] ID=18 | src=4 | dst=[SERVER] ID=1 | Mu=8
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.QueueImpl: +++ QueueImpl : Dequeued message with id=19
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.ServerImpl: +++ ServerImpl : Message dequeued : [Message] ID=19 | src=4 | dst=[SERVER] ID=1 | Mu=8
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.QueueImpl: +++ QueueImpl : Dequeued message with id=20
2026-05-13 20:30:58 INFO net.lecnam.rcp103.tp2.ServerImpl: +++ ServerImpl : Message dequeued : [Message] ID=20 | src=4 | dst=[SERVER] ID=1 | Mu=8
```

FIGURE 3 – Fin d’exécution et traces de fermeture de la méthode calcul_MM1_rate

15.6 Apport de cette validation

Ces captures d’écran complètent les tests unitaires et les extraits de code présentés précédemment. Elles prouvent que :

- le moteur `Engine.py` est bien exécuté ;
- la méthode `calcul_MM1_rate` est appelée ;
- les valeurs théoriques M/M/1 sont calculées ;

- le cas limite $\lambda = \mu$ est correctement détecté ;
- l’affichage **INFINI** est bien produit ;
- les logs permettent de suivre le début et la fin de l’exécution.

Cette validation donne donc une preuve concrète du bon fonctionnement du code pour la partie théorique M/M/1 demandée dans le TP.

16 Intégration avec le scheduler

16.1 Rappel du rôle du scheduler

Le scheduler reste responsable de l’organisation temporelle des événements. Il stocke les événements futurs et les restitue dans l’ordre chronologique.

Son interface minimale contient notamment :

```

1 class IScheduler(ABC):
2
3     @abstractmethod
4     def add_event(self, event: IEvent):
5         pass

```

Listing 37 – Extrait de IScheduler.py

16.2 Lien avec les nouveaux composants

Les composants introduits en séance 2 interagiront avec le scheduler de la manière suivante :

- le client génère des événements **SEND_MSG** ;
- la passerelle génère des événements **RECV_MSG** ;
- le serveur génère des événements **MSG_DEPT** ;
- la file d’attente influence la date de début de service d’un message ;
- le moteur extrait les événements du scheduler et déclenche les traitements associés.

16.3 Scénario conceptuel

Un scénario simple peut être décrit ainsi :

1. le client crée un message au temps t ;
2. un événement **SEND_MSG** est ajouté au scheduler ;
3. lorsque l’événement est traité, un événement **RECV_MSG** est programmé à $t + d$, où d est le délai de propagation ;
4. à la réception, si un serveur est libre, le message est traité immédiatement ;
5. sinon, le message est inséré dans **QueueImpl** ;
6. à la fin du service, un événement **MSG_DEPT** est traité ;
7. si la file contient un message, celui-ci est extrait et servi.

17 Préparation des métriques

17.1 Métriques attendues

Les métriques prévues dans le sujet sont :

- le nombre instantané de requêtes dans le système ;
- le nombre instantané de requêtes dans la file ;
- le nombre total d’arrivées ;
- le nombre total de départs ;
- le nombre de messages rejetés ;
- le nombre moyen de requêtes dans le système ;

- le nombre moyen de requêtes dans la file ;
- le temps moyen dans le système ;
- le temps moyen en file ;
- le débit.

17.2 Temps de séjour

Le timestamp du message permet de calculer le temps total passé dans le système :

$$W_{\text{système},i} = t_{\text{départ},i} - t_{\text{création},i}$$

Le temps moyen dans le système est alors :

$$\overline{W}_{\text{système}} = \frac{1}{N} \sum_{i=1}^N (t_{\text{départ},i} - t_{\text{création},i})$$

17.3 Nombre moyen de messages

Le nombre moyen de messages dans le système est une moyenne temporelle. Si N_i est le nombre de messages dans le système pendant l'intervalle Δt_i , alors :

$$\overline{N} = \frac{1}{T} \sum_i N_i \Delta t_i$$

Cette métrique nécessite de mettre à jour l'aire sous la courbe $N(t)$ à chaque événement.

17.4 Loi de Little

La loi de Little fournit une relation de cohérence entre le nombre moyen de messages, le débit et le temps moyen :

$$L = \lambda W$$

Dans la séance 3, cette loi pourra être utilisée pour vérifier les résultats expérimentaux du simulateur.

18 Lien avec les modèles de file d'attente

18.1 Modèle M/M/1

Le modèle M/M/1 correspond à :

- une arrivée markovienne ;
- un temps de service markovien ;
- un serveur unique ;
- une file de capacité infinie.

Pour ce modèle, les formules théoriques sont :

$$\rho = \frac{\lambda}{\mu}$$

$$L = \frac{\rho}{1 - \rho}$$

$$W = \frac{1}{\mu - \lambda}$$

La condition de stabilité est :

$$\rho < 1 \iff \lambda < \mu$$

18.2 Modèles M/M/1/K

Les modèles M/M/1/4 et M/M/1/8 ajoutent une capacité maximale K . Lorsque le système contient déjà K messages, toute nouvelle arrivée est rejetée.

La file `QueueImpl` devra donc permettre de détecter l'état plein du système.

18.3 Modèle M/M/3/8

Le modèle M/M/3/8 introduit trois serveurs partageant une même file. La classe `ServerImpl` doit donc pouvoir être instanciée plusieurs fois, et le moteur doit être capable de sélectionner un serveur libre.

La création de plusieurs serveurs dans `Engine.py` :

```
1 engine.create_servers(2)
```

prépare cette logique multi-serveurs.

19 Synthèse de la séance 2

La séance 2 a permis d'ajouter au simulateur les composants opérationnels nécessaires à la modélisation d'une file d'attente. Les principales avancées sont :

- l'introduction de la classe `ClientImpl` pour représenter les générateurs de messages ;
- l'introduction de la classe `QueueImpl` pour représenter la file d'attente ;
- l'introduction de la classe `ServerImpl` pour représenter les serveurs ;
- la mise en place de la classe `ConfigImpl` pour centraliser les paramètres ;
- la préparation des distributions aléatoires avec `Distribution` et `Poisson` ;
- l'évolution de `Engine.py` vers un rôle d'orchestrateur ;
- l'ajout de tests séparés pour les nouveaux composants ;
- la préparation des métriques qui seront exploitées lors de la séance 3.

Ces éléments complètent les fondations posées lors de la première séance. Le simulateur dispose maintenant des objets nécessaires pour représenter les entités principales d'un système client-file-serveur.

20 Conclusion

Cette deuxième séance constitue une étape intermédiaire importante dans la construction du simulateur à événements discrets. Après la mise en place des messages, événements et du scheduler lors de la séance 1, le projet introduit maintenant les composants qui décrivent le comportement opérationnel du système : clients, file d'attente et serveurs.

La classe `ClientImpl` prépare la génération de messages et d'événements d'arrivée. La classe `QueueImpl` permet de représenter les messages en attente selon une logique FIFO. La classe `ServerImpl` prépare la modélisation du traitement et des événements de départ. La classe `ConfigImpl` centralise les paramètres et le logging, tandis que les classes `Distribution` et `Poisson` préparent la génération aléatoire nécessaire aux modèles probabilistes.

Le moteur `Engine.py` commence à intégrer ces composants en créant plusieurs clients et plusieurs serveurs. Cette évolution prépare directement la séance 3, qui consistera à assembler le simulateur complet, à implémenter la passerelle, à générer les traces finales et à comparer les résultats expérimentaux aux formules théoriques des modèles M/M/1, M/M/1/K et M/M/c/K.

A Annexe A – ConfigImpl.py

```
1 #!/usr/bin/python3
2
3 #####
4 #
5 # PYTHONPATH=. /usr/bin/python3 net/lecnam/rcp103/tp2/ConfigImpl.py
6 #
7 # #####
8
9 import os
10 import logging
11 import logging.config
12
13 from net.lecnam.rcp103.SimulateurException import SimulateurException
14 from net.lecnam.rcp103.tp2.IConfig import IConfig
15
16 # Always load logging_config.py from the same directory as this file
17 config_path = os.path.join(os.path.dirname(__file__), "logging_config.cnf")
18 logging.config.fileConfig(config_path, defaults=None, disable_existing_loggers=False, encoding
19                             =None)
20
21 logger = logging.getLogger(__name__)
22 # https://docs.python.org/3/library/logging.html#logging-levels
23
24 # Class d'Implmentation
25 class ConfigImpl(IConfig):
26
27     SEED= 3 # correspond au Groupe 3
28     OUPUT_DIR = "RCP103_TP2_OUTPUTS"
29     LOG_CFG_FILE_PATH = "logging_config.cnf"
30
31     def __init__(self):
32         logger.debug(f"+++ ConfigImpl : START Constructor")
33         logger.debug(f"+++ ConfigImpl : END Constructor")
34         pass
35
36     def get_seed(self) -> int:
37         return self.SEED
38
39     def get_output_dir(self) -> str:
40         return self.OUPUT_DIR
41
42     def get_log_cfg_file_path(self) -> str:
43         return self.LOG_CFG_FILE_PATH
44
45     # --- Affichage ---
46     def print_config(self):
47         return(f"[Config] seed={self.SEED} | output_dir={self.OUPUT_DIR} | log_cfg_file_path={
48             self.LOG_CFG_FILE_PATH}")
```

Listing 38 – ConfigImpl.py

B Annexe B – ClientImpl.py

```
1 #!/usr/bin/python3
2
3 #####
4 #
5 # PYTHONPATH=. /usr/bin/python3 net/lecnam/rcp103/tp2/ClientImpl.py
6 #
7 # #####
8
9 from abc import abstractmethod
10 import datetime
11 import os
12 import platform
```



```

13 import queue
14 import string
15 import sys
16 import secrets
17 import traceback
18
19 import logging
20 import logging.config
21
22 # from scipy.stats import poisson
23 # https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.poisson.html
24
25 import numpy as np
26 import matplotlib.pyplot as plt
27
28 from net.lecnam.rcp103.tp2.IEvent import IEvent
29 from net.lecnam.rcp103.tp2.IMessage import IMessage
30 from net.lecnam.rcp103.tp2.EventType import EventType
31 from net.lecnam.rcp103.tp2.IClient import IClient
32 from net.lecnam.rcp103.tp2.ConfigImpl import ConfigImpl
33 from net.lecnam.rcp103.SimulateurException import SimulateurException
34 from net.lecnam.rcp103.tp2 import IQueue
35 from net.lecnam.rcp103.tp2.Poisson import Poisson
36 from net.lecnam.rcp103.tp2.Poisson import Distribution
37 from net.lecnam.rcp103.tp2.QueueImpl import QueueImpl
38 from net.lecnam.rcp103.tp2.IServer import IServer
39
40 cfg = ConfigImpl()
41 log_path = cfg.get_log_cfg_file_path()
42
43 # Always load logging_config.py from the same directory as this file
44 config_path = os.path.join(os.path.dirname(__file__), log_path)
45 logging.config.fileConfig(config_path, defaults=None, disable_existing_loggers=False, encoding
    =None)
46
47 logger = logging.getLogger(__name__)
48 # https://docs.python.org/3/library/logging.html#logging-levels
49
50 # Class d'Impl mentation
51 class ClientImpl(IClient):
52
53     message: IMessage
54     arrival_rate: int # arrival rate (lambda) of the Poisson distribution
55     queue: IQueue
56     client_id: int
57     destination: IServer
58
59     def __init__(self, arrival_rate: int, q:IQueue):
60         logger.debug(f"+++ ClientImpl : START Constructor")
61
62         seed = cfg.get_seed()
63         logger.debug(f"+++ ClientImpl : seed={seed}")
64         logger.debug(f"+++ ClientImpl : arrival_rate={arrival_rate}")
65         self.arrival_rate = arrival_rate
66
67         self.queue = q
68
69         logger.debug(f"+++ ClientImpl : END Constructor")
70
71     def send_message(self, msg: IMessage):
72         logger.debug(f"+++ ClientImpl : START send_message")
73         self.queue.enqueue(msg)
74         logger.debug(f"+++ ClientImpl : END send_message")
75
76     def set_client_id(self, id: int):
77         self.client_id = id
78
79     def get_client_id(self):
80         return self.client_id

```

```

81
82     def get_destination(self):
83         return self.destination
84
85     def set_destination(self, dst: IServer):
86         self.destination = dst
87
88     def get_arrival_rate(self):
89         return self.arrival_rate
90
91     def set_arrival_rate(self, rate: int):
92         self.arrival_rate = rate
93
94     def set_message(self, msg: IMessage):
95         self.message = msg
96
97     def get_message(self, msg: IMessage):
98         return self.message
99
100     # --- Affichage ---
101     def print_client(self):
102         logger.debug(f"+++ ClientImpl : START print_client")
103
104         # msg = self.message.print_message()
105         if not self.queue.is_empty():
106             q = self.queue.print_queue()
107         else:
108             q = "Empty"
109
110         srv = self.destination.print_server()
111         client = f"ClientID={self.client_id} | ArrivalRate={self.arrival_rate} | Queue={q} | Destination={srv}"
112         logger.debug(f"+++ ClientImpl : Client " + str(client))
113         logger.debug(f"+++ ClientImpl : END print_client")
114         return(client)

```

Listing 39 – ClientImpl.py

C Annexe C – QueueImpl.py

```

1 #!/usr/bin/python3
2
3 #####
4 #
5 # PYTHONPATH=. /usr/bin/python3 net/lecnam/rcp103/tp2/QueueImpl.py
6 #
7 # #####
8
9 import datetime
10 import os
11 import platform
12 import string
13 import sys
14 import secrets
15 import traceback
16
17 import logging
18 import logging.config
19
20 from net.lecnam.rcp103.tp2.IQueue import IQueue
21 from net.lecnam.rcp103.tp2.ConfigImpl import ConfigImpl
22 from net.lecnam.rcp103.SimulateurException import SimulateurException
23
24 import queue
25 import numpy as np
26
27 from net.lecnam.rcp103.tp2.IMessage import IMessage
28

```

```

29 cfg = ConfigImpl()
30 log_path = cfg.get_log_cfg_file_path()
31
32 # Always load logging_config.py from the same directory as this file
33 config_path = os.path.join(os.path.dirname(__file__), log_path)
34 logging.config.fileConfig(config_path, defaults=None, disable_existing_loggers=False, encoding
    =None)
35
36 logger = logging.getLogger(__name__)
37 # https://docs.python.org/3/library/logging.html#logging-levels
38
39 # Class d'Impl mentation
40 class QueueImpl(IQueue):
41
42     queue_size = -1 # If maxsize is <= 0, the queue size is infinite.
43     queue: list[IMessage]
44     # mu: int # service rate (mu) of the M/M/1 queue
45     # lam: int # arrival rate (lambda) of the Poisson distribution
46
47     def __init__(self):
48         logger.debug(f"+++ QueueImpl : START Constructor")
49         self.queue = []
50         logger.debug(f"+++ QueueImpl : END Constructor")
51
52     def enqueue(self, msg: IMessage):
53         logger.debug(f"+++ QueueImpl : START enqueue")
54         self.queue.append(msg)
55         logger.debug(f"+++ QueueImpl : END enqueue")
56
57     def dequeue(self):
58         logger.debug(f"+++ QueueImpl : START dequeue")
59         if len(self.queue) == 0:
60             logger.error(f"+++ QueueImpl : dequeue called on an empty queue")
61             return None
62         else:
63             msg = self.queue.pop(0) # [1, 2, 3, 4] -> pop(0) -> 1
64             logger.info(f"+++ QueueImpl : Dequeued message with id={msg.get_message_id()}")
65             logger.debug(f"+++ QueueImpl : END dequeue")
66             return msg
67
68     def count_messages(self):
69         logger.debug(f"+++ QueueImpl : START count_messages")
70         nb_msg = len(self.queue)
71         logger.debug(f"+++ QueueImpl : count_messages = {nb_msg}")
72         logger.debug(f"+++ QueueImpl : END count_messages")
73         return nb_msg
74
75     def is_empty(self):
76         #logger.debug(f"+++ QueueImpl : START is_empty")
77         empty = len(self.queue) == 0
78         #logger.debug(f"+++ QueueImpl : is_empty = {empty}")
79         #logger.debug(f"+++ QueueImpl : END is_empty")
80         return empty
81
82     """ --- Affichage de TOUS les messages --- """
83     def print_messages(self):
84         logger.debug(f"+++ QueueImpl : START print_messages")
85
86         if not self.is_empty():
87             for msg in self.queue:
88                 all_messages += msg.print_message()
89
90             logger.info(all_messages)
91             return all_messages
92         else:
93             logger.info(f"+++ QueueImpl : No messages in the queue to print.")
94             return None
95         logger.debug(f"+++ QueueImpl : END print_messages")

```

Listing 40 – QueueImpl.py

D Annexe D – ServerImpl.py

```
1 #!/usr/bin/python3
2
3 #####
4 #
5 # PYTHONPATH=. /usr/bin/python3 net/lecnam/rcp103/tp2/ServerImpl.py
6 #
7 # #####
8
9 import datetime
10 import os
11 import platform
12 import string
13 import sys
14 import secrets
15 import traceback
16
17 import logging
18 import logging.config
19
20 from time import time, sleep
21
22 import numpy as np
23
24 from net.lecnam.rcp103.tp2.IEvent import IEvent
25 from net.lecnam.rcp103.tp2.IMessage import IMessage
26 from net.lecnam.rcp103.tp2.EventType import EventType
27 from net.lecnam.rcp103.tp2.IClient import IClient
28 from net.lecnam.rcp103.tp2.IQueue import IQueue
29 from net.lecnam.rcp103.tp2.IServer import IServer
30 from net.lecnam.rcp103.tp2.ConfigImpl import ConfigImpl
31 from net.lecnam.rcp103.SimulateurException import SimulateurException
32
33 try:
34     cfg = ConfigImpl()
35     log_path = cfg.get_log_cfg_file_path()
36
37     # Always load logging_config.py from the same directory as this file
38     config_path = os.path.join(os.path.dirname(__file__), log_path)
39     logging.config.fileConfig(config_path, defaults=None, disable_existing_loggers=False,
40                               encoding=None)
41
42     logger = logging.getLogger(__name__)
43     # https://docs.python.org/3/library/logging.html#logging-levels
44 except Exception as e:
45     # Fallback to basic logging if config file fails
46     logging.basicConfig(level=logging.DEBUG, format='%(asctime)s - %(name)s - %(levelname)s -
47     %(message)s')
48     logger = logging.getLogger(__name__)
49     logger.error(f"Failed to load logging config from {config_path}: {e}")
50
51 # Class d'implémentation
52 class ServerImpl(IServer):
53
54     mu: int # service rate (mu) of the M/M/1 queue
55     server_id: int
56
57     queue: IQueue
58
59     def __init__(self, mu: int, server_id: int, queue: IQueue):
60         logger.debug(f"+++ ServerImpl : START Constructor")
61
62         self.mu = mu
63         self.server_id = server_id
64         self.queue = queue
65
66         logger.debug(f"+++ ServerImpl : END Constructor")
```

```

66
67 def get_queue(self) -> IQueue:
68     return self.queue
69
70 def set_queue(self, queue: IQueue):
71     self.queue = queue
72
73 def get_server_id(self):
74     return self.server_id
75
76 def set_server_id(self, server_id: int):
77     self.server_id = server_id
78
79 def listen(self):
80     logger.debug(f"+++ ServerImpl : START listen")
81
82     # Loop on Queue messages to dequeue and process them
83     while True:
84         if not self.queue.is_empty():
85             msg = self.queue.dequeue()
86             logger.info(f"+++ ServerImpl : Message dequeued : {msg.print_message()}")
87             # Process the message (e.g., simulate service time, send response, etc.)
88             # For simplicity, we just print the message here
89         else:
90             #logger.debug(f"+++ ServerImpl : Queue is empty, waiting for messages...")
91             # Optionally, add a sleep here to avoid busy waiting
92             sleep(0.1)
93
94     logger.debug(f"+++ ServerImpl : END listen")
95
96 def get_server_id():
97     return self.server_id
98
99 def set_server_id(self, server_id: int):
100     self.server_id = server_id
101
102 # --- Affichage ---
103 def print_server(self):
104     logger.debug(f"+++ ServerImpl : START listen")
105     # q = self.queue.print_messages()
106     srv = f"[SERVER] ID={self.server_id} | Mu={self.mu}"
107     logger.debug(f"+++ ServerImpl : END listen")
108     return(srv)

```

Listing 41 – ServerImpl.py

E Annexe E – Distribution.py

```

1 #!/usr/bin/python3
2
3 # PYTHONPATH=. /usr/bin/python3 net/lecnam/rcp103/Distribution.py
4
5 import os
6 import traceback
7 import logging
8 import logging.config
9
10 import numpy as np
11
12 from net.lecnam.rcp103.tp2.ConfigImpl import ConfigImpl
13
14 cfg = ConfigImpl()
15 log_path = cfg.get_log_cfg_file_path()
16 config_path = os.path.join(os.path.dirname(__file__), log_path)
17 logging.config.fileConfig(config_path, defaults=None, disable_existing_loggers=False, encoding
18                             =None)
19 seed = cfg.get_seed()
20 rng = np.random.default_rng(seed=seed)

```

```

20 logger = logging.getLogger(__name__)
21
22 # Classe parente pour les diff rentes distributions
23 class Distribution:
24     """Classe de base repr sentant une distribution statistique."""
25
26     def __init__(self, name: str, rng):
27         logger.debug(f"+++ Distribution __init__ : START")
28         self.name = name
29         self.rng = rng
30         logger.debug(f"+++ Distribution __init__ : END")
31
32     def generate(self, n: int) -> np.ndarray:
33         raise NotImplementedError("La m thode generate() doit tre impl ment e dans la
34 sous-classe.")
35
36     def __repr__(self):
37         return f"{self.__class__.__name__}(name={self.name!r})"

```

Listing 42 – Distribution.py

F Annexe F – Poisson.py

```

1 #!/usr/bin/python3
2
3 # PYTHONPATH=. /usr/bin/python3 net/lecnam/rcp103/Poisson.py
4
5 import os
6 import traceback
7 import logging
8 import logging.config
9
10 import numpy as np
11
12 from net.lecnam.rcp103.tp2.ConfigImpl import ConfigImpl
13 from net.lecnam.rcp103.tp2.Distribution import Distribution
14
15 cfg = ConfigImpl()
16 log_path = cfg.get_log_cfg_file_path()
17 config_path = os.path.join(os.path.dirname(__file__), log_path)
18 logging.config.fileConfig(config_path, defaults=None, disable_existing_loggers=False, encoding
19 =None)
20 logger = logging.getLogger(__name__)
21
22 class Poisson(Distribution):
23     """Distribution de Poisson."""
24
25     seed = cfg.get_seed()
26     rng = np.random.default_rng(seed=seed)
27
28     def __init__(self, rng, lam: float):
29         logger.debug(f"+++ Poisson __init__ : START")
30         logger.info(f"+++ Poisson __init__ seed={self.seed}, lam={lam}")
31         super().__init__(f"Poisson ( ={lam})", rng)
32         self.lam = lam
33         logger.debug(f"+++ Poisson __init__ : END")
34
35     def generate(self, n: int) -> np.ndarray:
36         return self.rng.poisson(self.lam, size=n)

```

Listing 43 – Poisson.py

G Annexe G – Extrait de Engine.py

```

1 #!/usr/bin/python3

```

```

2
3 # Commande pour lancer le programme :
4 # PYTHONPATH=. python3 net/lecnam/rcp103/tp2/Engine.py
5 # en Linux/WSL: python3 -m net.lecnam.rcp103.tp2.Engine
6 # sous Windows/PowerShell: python Engine.py
7
8 from net.lecnam.rcp103.tp2.MessageImpl import MessageImpl
9 from net.lecnam.rcp103.tp2.EventImpl import EventImpl
10 from net.lecnam.rcp103.tp2.SchedulerImpl import SchedulerImpl
11 from net.lecnam.rcp103.tp2.IScheduler import IScheduler
12 from net.lecnam.rcp103.tp2.IServer import IServer
13 from net.lecnam.rcp103.tp2.IClient import IClient
14 from net.lecnam.rcp103.tp2.IQueue import IQueue
15 from net.lecnam.rcp103.tp2.ServerImpl import ServerImpl
16 from net.lecnam.rcp103.tp2.ClientImpl import ClientImpl
17 from net.lecnam.rcp103.tp2.QueueImpl import QueueImpl
18 from net.lecnam.rcp103.tp2.Poisson import Poisson
19 from net.lecnam.rcp103.tp2.Poisson import Distribution
20 from net.lecnam.rcp103.tp2.ConfigImpl import ConfigImpl
21
22 import numpy as np
23
24 import logging
25 import logging.config
26 import os
27 import platform
28 import string
29 import sys
30
31 # Always load logging_config.py from the same directory as this file
32 config_path = os.path.join(os.path.dirname(__file__), "logging_config.cnf")
33 logging.config.fileConfig(config_path, defaults=None, disable_existing_loggers=False, encoding
    =None)
34
35 logger = logging.getLogger(__name__)
36 # https://docs.python.org/3/library/logging.html#logging-levels
37
38 class Engine:
39
40     nb_clients: int
41     nb_servers: int
42     lambda_arrival_rate: list
43     service_rate: int
44     simulation_time: float
45     clients: list
46     servers: list[IServer]
47     scheduler: IScheduler
48     queue: IQueue
49
50     def __init__(self):
51         logger.debug("+++ Engine : START __init__ ...")
52         self.scheduler = SchedulerImpl()
53         self.nb_clients = 0
54         self.nb_servers = 0
55         self.service_rate = 8
56         self.lambda_arrival_rate = [4, 6, 8 , 12] # arrival rate (lambda) of the Poisson
distribution
57         self.simulation_time = 10.0
58         self.clients = []
59         self.servers = []
60         self.queue = QueueImpl()
61
62         logger.debug("+++ Engine : START __init__ ...")
63
64     def create_clients(self, n):
65         logger.debug("+++ Engine : START create_clients ...")
66         self.nb_clients = n
67         self.clients = []
68

```

```

69     logger.info(f"+++ Engine : n=" + str(n))
70     i=0
71     for i in range(1, n + 1):
72         logger.debug(f"+++ Engine : i=" + str(i))
73         client = ClientImpl(arrival_rate=self.lambdas[i-1], q=self.queue)
74         client.set_client_id(i)
75         client.set_destination(self.servers[0])
76         self.clients.append(client)
77         pretty_client = client.print_client()
78         logger.info("+++ Engine : Clients cr   s   : " + str(pretty_client))
79         i+=1
80
81     logger.debug("+++ Engine : END create_clients ...")
82
83     def create_servers(self, n):
84         logger.debug("+++ Engine : START create_servers ...")
85         self.nb_servers = n
86         logger.debug("+++ Engine : START about to create n servers, n=" + str(n))
87         self.servers = []
88         for i in range(1, n + 1):
89             srv = ServerImpl(server_id=i, mu=self.service_rate, queue=self.queue)
90             self.servers.append(srv)
91             pretty_srv = srv.print_server()
92             logger.info("+++ Engine : Server created:" + str(pretty_srv))
93
94         logger.debug("+++ Engine : END create_servers ...")
95
96     def print_trace_header(self):
97         print("\n--- TRACE ---")
98         print(f"{'time':<8} {'node':<5} {'event':<6} {'src':<5} {'dst':<5} {'msgID':<5}")
99         print("-" * 45)
100
101     def generate_trace(self, event):
102         msg = event.get_message()
103
104         event_type = event.get_event_type()
105
106         if event_type == "SEND_MSG":
107             node = msg.get_source()
108             event_name = "SEND"
109         elif event_type == "RECV_MSG":
110             node = 0
111             event_name = "RECV"
112         elif event_type == "MSG_DEPT":
113             node = 0
114             event_name = "DEPT"
115         else:
116             node = 0
117             event_name = event_type
118
119         print(
120             f"{event.get_event_time():<8.3f} "
121             f"{node:<5} "
122             f"{event_name:<6} "
123             f"{msg.get_source():<5} "
124             f"{msg.get_destination():<5} "
125             f"{msg.get_message_id():<5}"
126         )
127
128     def run(self):
129         self.print_trace_header()
130
131         while self.scheduler.has_events():
132             event = self.scheduler.get_event()
133             self.generate_trace(event)
134
135     def test_message(self):
136         print("\n--- TEST MESSAGE ---")
137         msg = MessageImpl(1, 1, 0, 0.0)

```



```

138         msg.print_message()
139
140     def test_event(self):
141         print("\n--- TEST EVENT ---")
142         msg = MessageImpl(1, 1, 0, 0.0)
143         event = EventImpl(1, msg, "SEND_MSG", 1.567)
144         prettyEvt = event.print_event()
145         print("+++ Pretty print = " + str(prettyEvt))
146
147     def test_scheduler(self):
148         print("\n--- TEST SCHEDULER ---")
149
150         msg1 = MessageImpl(1, 1, 0, 0.0)
151         msg2 = MessageImpl(2, 3, 0, 0.0)
152
153         e1 = EventImpl(1, msg1, "SEND_MSG", 1.202)
154         e2 = EventImpl(2, msg1, "RECV_MSG", 1.916)
155         e3 = EventImpl(3, msg2, "SEND_MSG", 2.320)
156         e4 = EventImpl(4, msg2, "RECV_MSG", 2.391)
157         e5 = EventImpl(5, msg1, "MSG_DEPT", 4.572)
158         e6 = EventImpl(6, msg2, "MSG_DEPT", 5.916)
159
160         self.scheduler.add_event(e5)
161         self.scheduler.add_event(e1)
162         self.scheduler.add_event(e3)
163         self.scheduler.add_event(e6)
164         self.scheduler.add_event(e2)
165         self.scheduler.add_event(e4)
166
167         self.scheduler.print_scheduler()
168         logger.debug("+++ Engine : print_scheduler ...")
169         logger.info(self.scheduler.print_scheduler())
170
171     def run_tests(self):
172         logger.debug("+++ Engine : START run_tests ...")
173         # self.test_message()
174         # self.test_event()
175         # self.test_scheduler()
176         self.run_simulationMM1()
177
178         print("\n--- RUN SIMULATION ---")
179         #self.run()
180         logger.debug("+++ Engine : END run_tests ...")
181
182     def run_simulationMM1(self):
183         logger.debug("+++ Engine : START run_simulationMM1 ...")
184
185         # Iterate on servers and trigger listen() method to process messages in the queue
186         for server in self.servers:
187             pretty_srv = server.print_server()
188             logger.info("+++ Engine run_simulationMM1: server about to start listening:" + str
189 (pretty_srv))
189             server.listen()
190
191         logger.debug("+++ Engine : END run_simulationMM1 ...")
192
193     def calcul_MM1_rate(self):
194         logger.debug("+++ Engine : START calcul_MM1_rate ...")
195         print("\n--- TRACE ---")
196         print(f'{'Lambda'} \t \t {'Rho'} \t \t {'L'} \t \t {'W'}')
197
198         # Iterate lambda_arrival_rate and calculate L, W, rho for each lambda and print the
199         # results
200         for lam in self.lambda_arrival_rate:
201             rho = lam / self.service_rate
202             try:
203                 # self.lambda_arrival_rate = [4, 6, 8 , 12]
204                 L = rho / (1 - rho)
205                 W = 1 / (self.service_rate - lam)

```

```

205         print(f"{lam:<8.3f} \t {rho:<8.3f} \t {L:<8.3f} \t{W:<8.3f}")
206         except ZeroDivisionError as e:
207             print(f"{lam:<8.3f} \t {rho:<8.3f} \t {"INFINI"} \t {"INFINI"}")
208             logger.error("Division by zero error in calcul_MM1_rate")
209     print("-" * 45)
210     logger.debug("+++ Engine : END calcul_MM1_rate ...")
211
212 if __name__ == "__main__":
213     logger.debug("+++ Engine : Main START ...")
214     engine = Engine()
215     engine.create_servers(2)
216     engine.create_clients(4)
217
218     cfg = ConfigImpl()
219     seed = cfg.get_seed()
220     rng = np.random.default_rng(seed=seed)
221
222     engine.calcul_MM1_rate()
223
224     # Loop on each client to send a message to the server
225     i=1
226     ts = 0.0
227     for client in engine.clients:
228         fish = Poisson(rng=rng, lam=client.get_arrival_rate())
229         for _ in range(5):
230             logger.debug("+++ Engine : i=" + str(i))
231             msg = MessageImpl(i, client.get_client_id(), client.get_destination(), ts+0.10)
232             message = msg.print_message()
233             logger.debug("+++ Engine : message = " , {message})
234             inter_arrival_time = fish.generate(1)[0] / 1000.0 # Convert ms to seconds
235             ts += inter_arrival_time
236             msg.set_timestamp(ts)
237             client.send_message(msg)
238             i+=1
239
240     engine.run_tests()
241     logger.debug("+++ Engine : Main END ...")

```

Listing 44 – Engine.py

H Annexe H – ClientImplTest.py

```

1  #!/usr/bin/python3
2
3  # Commande pour lancer le programme :
4  # PYTHONPATH=. python3 net/lecnam/rcp103/tp2/ClientImplTest.py
5  # en Linux/WSL: python3 -m net.lecnam.rcp103.tp2.ClientImplTest
6  # sous Windows/PowerShell: python ClientImplTest.py
7
8  from datetime import datetime
9  import traceback
10 import logging
11 import os
12
13 from net.lecnam.rcp103.tp2.ConfigImpl import ConfigImpl
14 from net.lecnam.rcp103.tp2.ClientImpl import ClientImpl
15 from net.lecnam.rcp103.tp2.ServerImpl import ServerImpl
16 from net.lecnam.rcp103.tp2.QueueImpl import QueueImpl
17
18 try:
19     cfg = ConfigImpl()
20     log_path = cfg.get_log_cfg_file_path()
21
22     # Always load logging_config.py from the same directory as this file
23     config_path = os.path.join(os.path.dirname(__file__), log_path)
24     logging.config.fileConfig(config_path, defaults=None, disable_existing_loggers=False,
25                               encoding=None)
26

```

```

26 logger = logging.getLogger(__name__)
27 # https://docs.python.org/3/library/logging.html#logging-levels
28
29 logger.debug(f"+++ ClientImplTest : START")
30 impl = ClientImpl(6)
31 queue = QueueImpl()
32 srv = ServerImpl(server_id=21, mu=8, queue=queue)
33
34 impl.set_client_id(42)
35 impl.set_destination(srv)
36 pretty_print = impl.print_client()
37
38 logger.info(f"+++ ClientImplTest : client = {pretty_print}")
39
40 logger.debug(f"+++ ClientImplTest : END")
41
42 except RuntimeError as error:
43     logger.error("Erreur au Runtime dans ClientImplTest")
44     print(f"A {type(error).__name__} has occurred.")
45     exit(42)
46 except Exception as exception :
47     logger.error(f"Exception dans ClientImplTest/Main")
48     print(f"A {type(exception).__name__} has occurred.")
49     print("")
50     traceback.print_exc()
51     print("")
52     print(exception)
53     exit(42)

```

Listing 45 – ClientImplTest.py

I Annexe I – QueueImplTest.py

```

1 #!/usr/bin/python3
2
3 # Commande pour lancer le programme :
4 # PYTHONPATH=. python3 net/lecnam/rcp103/tp2/QueueImplTest.py
5 # en Linux/WSL: python3 -m net.lecnam.rcp103.tp2.QueueImplTest
6 # sous Windows/PowerShell: python QueueImplTest.py
7
8 from datetime import datetime
9 import secrets
10 import traceback
11 import logging
12 import os
13
14 from net.lecnam.rcp103.tp2.ConfigImpl import ConfigImpl
15 from net.lecnam.rcp103.tp2.QueueImpl import QueueImpl
16 # from net.lecnam.rcp103.tp2.Poisson import Poisson
17 from net.lecnam.rcp103.tp2 import Distribution
18
19 import numpy as np
20
21
22 try:
23     cfg = ConfigImpl()
24     log_path = cfg.get_log_cfg_file_path()
25
26     # Always load logging_config.py from the same directory as this file
27     config_path = os.path.join(os.path.dirname(__file__), log_path)
28     logging.config.fileConfig(config_path, defaults=None, disable_existing_loggers=False,
29                               encoding=None)
30
31     logger = logging.getLogger(__name__)
32     # https://docs.python.org/3/library/logging.html#logging-levels
33
34     logger.debug(f"+++ QueueImplTest : START")

```

```

35     seed = cfg.get_seed()
36     rng = np.random.default_rng(seed=seed)
37
38     #fish = Poisson(rng=secrets.SystemRandom(), lam=4)
39     impl = QueueImpl()
40     q = impl.print_messages()
41
42     logger.info(f"+++ QueueImplTest : q = {q}")
43
44     logger.debug(f"+++ QueueImplTest : END")
45
46 except RuntimeError as error:
47     logger.error("Erreur au Runtime dans QueueImplTest")
48     print(f"A {type(error).__name__} has occurred.")
49     exit(42)
50 except Exception as exception :
51     logger.error(f"Exception dans QueueImplTest/Main")
52     print(f"A {type(exception).__name__} has occurred.")
53     print("")
54     traceback.print_exc()
55     print("")
56     print(exception)
57     exit(42)

```

Listing 46 – QueueImplTest.py

J Annexe J – ServerImplTest.py

```

1  #!/usr/bin/python3
2
3  # Commande pour lancer le programme :
4  # PYTHONPATH=. python3 net/lecnam/rcp103/tp2/ServerImplTest.py
5  # en Linux/WSL: python3 -m net.lecnam.rcp103.tp2.ServerImplTest
6  # sous Windows/PowerShell: python ServerImplTest.py
7
8  from datetime import datetime
9
10 import traceback
11 import logging
12 import os
13
14 from net.lecnam.rcp103.tp2.ConfigImpl import ConfigImpl
15 from net.lecnam.rcp103.tp2.ServerImpl import ServerImpl
16
17 from net.lecnam.rcp103.tp2.MessageImpl import MessageImpl
18 from net.lecnam.rcp103.tp2.IServer import IServer
19 from net.lecnam.rcp103.tp2.IQueue import IQueue
20 from net.lecnam.rcp103.tp2.ServerImpl import ServerImpl
21 from net.lecnam.rcp103.tp2.QueueImpl import QueueImpl
22 from net.lecnam.rcp103.tp2.Poisson import Poisson
23 from net.lecnam.rcp103.tp2.Poisson import Distribution
24
25 try:
26     cfg = ConfigImpl()
27     log_path = cfg.get_log_cfg_file_path()
28
29     # Always load logging_config.py from the same directory as this file
30     config_path = os.path.join(os.path.dirname(__file__), log_path)
31     logging.config.fileConfig(config_path, defaults=None, disable_existing_loggers=False,
32                               encoding=None)
33
34     logger = logging.getLogger(__name__)
35     # https://docs.python.org/3/library/logging.html#logging-levels
36
37     logger.debug(f"+++ ServerImplTest : START")
38     queue = QueueImpl()
39     impl = ServerImpl(8, 1, queue)

```

```

40 pretty_srv = impl.print_server()
41 logger.debug("+++ ServerImplTest : Server created:" + str(pretty_srv))
42
43 impl.listen()
44
45 logger.debug(f"+++ ServerImplTest : END")
46
47 except RuntimeError as error:
48     logger.error(f"Erreur au Runtime dans ServerImplTest")
49     print(f"A {type(error).__name__} has occurred.")
50     exit(42)
51 except Exception as exception :
52     logger.error(f"Exception dans ServerImplTest/Main")
53     print(f"A {type(exception).__name__} has occurred.")
54     print("")
55     traceback.print_exc()
56     print("")
57     print(exception)
58     exit(42)

```

Listing 47 – ServerImplTest.py

K Annexe K – ConfigImplTest.py

```

1  #!/usr/bin/python3
2
3  # Commande pour lancer le programme :
4  # PYTHONPATH=. python3 net/lecnam/rcp103/tp2/ConfigImplTest.py
5  # en Linux/WSL: python3 -m net.lecnam.rcp103.tp2.ConfigImplTest
6  # sous Windows/PowerShell: python ConfigImplTest.py
7
8  from datetime import datetime
9  import traceback
10 import logging
11 import os
12
13 from net.lecnam.rcp103.tp2.ConfigImpl import ConfigImpl
14
15 # Always load logging_config.py from the same directory as this file
16 config_path = os.path.join(os.path.dirname(__file__), "logging_config.cnf")
17 logging.config.fileConfig(config_path, defaults=None, disable_existing_loggers=False, encoding
    =None)
18
19 logger = logging.getLogger(__name__)
20 # https://docs.python.org/3/library/logging.html#logging-levels
21
22 try:
23     logger.debug(f"+++ ConfigImplTest : START")
24     cfg = ConfigImpl()
25     out_dir = cfg.get_output_dir()
26     seed = cfg.get_seed()
27     log_path = cfg.get_log_cfg_file_path()
28     conf = cfg.print_config()
29     logger.info(f"+++ ConfigImplTest : seed= {seed}")
30     logger.info(f"+++ ConfigImplTest : out_dir= {out_dir}")
31     logger.info(f"+++ ConfigImplTest : log_cfg_file_path= {log_path}")
32     logger.info(f"+++ ConfigImplTest : Config created: {conf}")
33     logger.debug(f"+++ ConfigImplTest : END")
34
35 except RuntimeError as error:
36     logger.error("Erreur au Runtime dans ConfigImplTest")
37     print(f"A {type(error).__name__} has occurred.")
38     exit(42)
39 except Exception as exception :
40     logger.error(f"Exception dans ConfigImplTest/Main")
41     print(f"A {type(exception).__name__} has occurred.")
42     print("")
43     traceback.print_exc()

```

```

44     print("")
45     print(exception)
46     exit(42)

```

Listing 48 – ConfigImplTest.py

L Annexe L – EventImpl.py

```

1  #!/usr/bin/python3
2
3  # PYTHONPATH=. /usr/bin/python3 net/lecnam/rcp103/tp2/EventImpl.py
4
5  # rapport      r diger: overleaf.com
6
7  #####
8  #
9  # pre-req: in VSCode install extension 'Microsoft Python Environments Extension' ('Python
10 # Environment Manager' is deprecated)
11 # https://scipy.org/install/: sudo apt-get install python3-scipy
12 # test in shell with : pip list
13
14 # Install on Windows:
15 # c:\python314\python.exe -m pip install matplotlib
16
17 # https://matplotlib.org/stable/install/index.html
18 # sudo apt upgrade
19 # sudo apt install python3-matplotlib
20 # pip install matplotlib
21 # python3 -m pip install -U pip
22 # python3 -m pip install -U matplotlib
23
24 # To manually trigger a refresh in VSCode:
25 # Open the Command Palette (Cmd+Shift+P or Ctrl+Shift+P)
26 # Run Python Environments: Refresh All Environment Managers
27 # #####
28
29 import datetime
30 import os
31 import platform
32 import string
33 import sys
34 import secrets
35 import traceback
36
37 import logging
38 import logging.config
39
40 from net.lecnam.rcp103.SimulateurException import SimulateurException
41
42 from scipy.stats import poisson
43 # https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.poisson.html
44
45
46 import numpy as np
47
48
49 import matplotlib.pyplot as plt
50
51 import scipy
52
53 from net.lecnam.rcp103.tp2.IEvent import IEvent
54 from net.lecnam.rcp103.tp2.IMessage import IMessage
55 from net.lecnam.rcp103.tp2.EventType import EventType
56
57 # print(scipy.__version__)
58
59 # Always load logging_config.py from the same directory as this file

```

```

60 config_path = os.path.join(os.path.dirname(__file__), "logging_config.cnf")
61 logging.config.fileConfig(config_path, defaults=None, disable_existing_loggers=False, encoding
    =None)
62
63 logger = logging.getLogger(__name__)
64 # https://docs.python.org/3/library/logging.html#logging-levels
65
66 # Class d'Implmentation
67 class EventImpl(IEvent):
68
69     seed= 3 # correspond au Groupe 3
70     OUPUT_DIR = "RCP103_TP2_OUTPUTS"
71
72     eventID: int
73     message: IMessage
74     eventType: string # EventType SEND_MSG, RECV_MSG, MSG_DEPT
75     eventTime: float # horodatage de l'vnement
76
77     def __init__(self, eventID: int, message: IMessage, eventType: string, eventTime: float):
78         self.eventID = eventID
79         self.message = message
80         self.eventTime = eventTime
81
82         if eventType not in [e.name for e in EventType]:
83             logger.error(f"+++ EventImpl __init__ Invalid event type in Constructor: {
eventType}. Must be one of: {[e.name for e in EventType]}")
84             raise ValueError(f"Invalid event type. Must be one of: {[e.name for e in EventType
85             ]}")
86         self.eventType = eventType
87
88     def get_event_time(self) -> datetime.datetime:
89         return self.eventTime
90
91     def get_event_type(self) -> string:
92         return self.eventType
93
94     def set_event_time(self, time: datetime):
95         self.eventTime = time
96
97     def set_event_type(self, type: string):
98         try:
99             logger.debug(f"+++ EventImpl : START setEventType ...")
100             if type not in [e.name for e in EventType]:
101                 logger.error(f"Invalid event type: {type}. Must be one of: {[e.name for e in
102                 EventType]}")
103                 raise ValueError(f"Invalid event type. Must be one of: {[e.name for e in
104                 EventType]}")
105             self.eventType = type
106             logger.debug(f"+++ EventImpl : evtType = " + str(type))
107             logger.debug(f"+++ EventImpl : END setEventType ...")
108         except KeyError:
109             raise ValueError(f"Invalid event type: {type}. Must be one of: {[e.name for e in
110             EventType]}")
111
112     def get_message(self) -> IMessage:
113         return self.message
114
115     def set_message(self, message: IMessage):
116         self.message = message
117
118     def get_event_id(self) -> int:
119         return self.eventID
120
121     def set_event_id(self, id: int):
122         self.eventID = id
123
124     def print_event(self):
125         msg = self.get_message().print_message()

```

```

123     logger.debug(f"+++ EventImpl : START print_event msg={msg}")
124     return(f"Event ID: {self.eventID}, Type: {self.eventType}, Time: {self.eventTime},
    Message: {msg}\n")

```

Listing 49 – EventImpl.py

M Annexe M – EventImplMockTest.py

```

1  #!/usr/bin/python3
2  # PYTHONPATH=. python3 net/lecnam/rcp103/tp2/EventImplMockTest.py
3  # Use unittest.mock to create a mock SimulateurImpl for Mock testing
4  # https://docs.python.org/3/library/unittest.mock.html
5
6  import traceback
7  import logging
8
9  import unittest
10 from unittest.mock import MagicMock
11 from net.lecnam.rcp103.tp2.EventImpl import EventImpl
12 from net.lecnam.rcp103.tp2.IEvent import IEvent
13 from net.lecnam.rcp103.tp2.IMessage import IMessage
14 from net.lecnam.rcp103.tp2.MessageImpl import MessageImpl
15 from net.lecnam.rcp103.tp2.EventType import EventType
16
17 class EventImplMockTest(unittest.TestCase):
18
19     def test_create_class_call_method(self):
20         # Create a mock instance of EventImpl
21         mock_sim = MagicMock(spec=EventImpl)
22
23         # Example: set return value for calcul
24         msg = MessageImpl(1, "Alice", "Bob", 1.64)
25         mock_sim.get_message.return_value = msg
26
27         mock_sim.get_event_time.return_value = 1.42
28         mock_sim.get_event_type.return_value = "SEND_MSG" # "Fake Foo Event"
29         mock_sim.get_event_id.return_value = 42
30
31         # Use the mock in your test
32         t = mock_sim.get_event_time()
33         type = mock_sim.get_event_type()
34         id = mock_sim.get_event_id()
35
36         print("Mocked id:", id)
37         print("Mocked type:", type)
38         print("Mocked t:", t)
39
40         # You can also assert calls
41         mock_sim.get_event_time.assert_called_once()
42         self.assertEqual(t, 1.42)
43         self.assertEqual(type, "SEND_MSG")
44         self.assertEqual(id, 42)
45
46 if __name__ == '__main__':
47     unittest.main()

```

Listing 50 – EventImplMockTest.py

N Annexe N – EventImplTest.py

```

1  #!/usr/bin/python3
2
3  # Commande pour lancer le programme :
4  # PYTHONPATH=. python3 net/lecnam/rcp103/tp2/EventImplTest.py
5  # en Linux/WSL: python3 -m net.lecnam.rcp103.tp2.EventImplTest

```



```

6 # sous Windows/PowerShell: python EventImplTest.py
7
8 from datetime import datetime
9 import traceback
10 import logging
11
12 from net.lecnam.rcp103.tp2.EventImpl import EventImpl
13 from net.lecnam.rcp103.tp2.IEvent import IEvent
14 from net.lecnam.rcp103.tp2.IMessage import IMessage
15 from net.lecnam.rcp103.tp2.MessageImpl import MessageImpl
16 from net.lecnam.rcp103.tp2.EventType import EventType
17
18 try:
19     print("+++ START EventImpl fire")
20     msg = MessageImpl(1, "Alice", "Bob", 1.13)
21     impl = EventImpl(1, msg, "SEND_MSG", 1.13) # SEND_MSG_ZZZ will force test to fail
22     horodatage = impl.get_event_time()
23     print("+++ result = " + str(horodatage))
24     res = impl.print_event()
25     print("+++ pretty print = " + str(res))
26     print("+++ END EventImpl ignite")
27
28 except RuntimeError as error:
29     print("Erreur au Runtime dans EventImpl")
30     print(f"A {type(error).__name__} has occurred.")
31     exit(42)
32 except Exception as exception :
33     print("Exception dans EventImpl/Main")
34     print(f"A {type(exception).__name__} has occurred.")
35     print("")
36     traceback.print_exc()
37     print("")
38     print(exception)
39     exit(42)

```

Listing 51 – EventImplTest.py

O Annexe O – EventType.py

```

1 from enum import Enum
2
3 ## EventType      SEND_MSG, RECV_MSG, MSG_DEPT
4
5 class EventType(Enum):
6     SEND_MSG = 1
7     RECV_MSG = 2
8     MSG_DEPT = 3

```

Listing 52 – EventType.py

P Annexe P – GatewayImpl.py

```

1 # !/usr/bin/python3
2
3 #####
4 #
5 # PYTHONPATH=. /usr/bin/python3 net/lecnam/rcp103/tp2/GatewayImpl.py
6 #
7 # #####
8
9 import datetime
10 import os
11 import platform
12 import string
13 import sys

```

```

14 import secrets
15 import traceback
16
17 import logging
18 import logging.config
19
20 from net.lecnam.rcp103.tp2.IEvent import IEvent
21 from net.lecnam.rcp103.tp2.IMessage import IMessage
22 from net.lecnam.rcp103.tp2.EventType import EventType
23 from net.lecnam.rcp103.tp2.IClient import IClient
24 from net.lecnam.rcp103.tp2.IQueue import IQueue
25 from net.lecnam.rcp103.tp2.IServer import IServer
26 from rcp103.net.lecnam.rcp103.tp2 import IGateway
27 from net.lecnam.rcp103.tp2.ConfigImpl import ConfigImpl
28 from net.lecnam.rcp103.SimulateurException import SimulateurException
29
30 cfg = ConfigImpl()
31 log_path = cfg.get_log_cfg_file_path()
32
33 # Always load logging_config.py from the same directory as this file
34 config_path = os.path.join(os.path.dirname(__file__), log_path)
35 logging.config.fileConfig(config_path, defaults=None, disable_existing_loggers=False, encoding
    =None)
36
37 logger = logging.getLogger(__name__)
38 # https://docs.python.org/3/library/logging.html#logging-levels
39 # Class d'Impl mentation
40 class GatewayImpl(IGateway):
41
42     queue: IQueue
43     server: IServer
44
45     def get_queue(self) -> IQueue:
46         return self.queue
47
48     def set_queue(self, queue: IQueue):
49         self.queue = queue
50
51     def get_server(self) -> IServer:
52         return self.server
53
54     def set_server(self, server: IServer):
55         self.server = server
56
57     def __init__(self, queue: IQueue, server: IServer):
58         logger.debug(f"+++ GatewayImpl : START Constructor")
59         self.queue = queue
60         self.server = server
61         logger.debug(f"+++ GatewayImpl : END Constructor")
62
63     # --- Affichage ---
64     def print_gateway(self):
65         logger.debug(f"+++ GatewayImpl : START print_gateway")
66         all_msg = self.get_queue().print_messages()
67         logger.debug(f"+++ GatewayImpl : Queue messages = {all_msg}")
68         srv = self.server.print_server()
69         logger.debug(f"+++ GatewayImpl : SERVER = {srv}")
70         logger.debug(f"+++ GatewayImpl : END print_gateway")
71         return(f"Server: {srv}, ALL Messages: {all_msg}\n")

```

Listing 53 – GatewayImpl.py

Q Annexe R – IClient.py

```

1
2 # Interface
3 from abc import ABC, abstractmethod
4

```

```

5 from net.lecnam.rcp103.tp2.IMessage import IMessage
6 from net.lecnam.rcp103.tp2.IServer import IServer
7
8
9 class IClient(ABC):
10
11     @abstractmethod
12     def get_destination():
13         pass
14
15     # Set Queue name
16     @abstractmethod
17     def set_destination(dst: IServer):
18         pass
19
20     @abstractmethod
21     def get_arrival_rate():
22         pass
23
24     @abstractmethod
25     def set_arrival_rate(rate: int):
26         pass
27
28     @abstractmethod
29     def send_message(msg: IMessage):
30         pass
31
32     @abstractmethod
33     def print_client():
34         pass
35
36     @abstractmethod
37     def set_message(msg: IMessage):
38         pass
39
40     @abstractmethod
41     def get_message(msg: IMessage):
42         pass
43
44     @abstractmethod
45     def set_client_id(id: int):
46         pass
47
48     @abstractmethod
49     def get_client_id():
50         pass
51
52     '''
53     @abstractmethod
54     def foo(x: int, y: int):
55         pass
56     '''

```

Listing 54 – IClient.py

R Annexe S – IConfig.py

```

1
2 # Interface
3 from abc import ABC, abstractmethod
4
5 from net.lecnam.rcp103 import logging_config
6
7 class IConfig(ABC):
8
9     @abstractmethod
10     def get_seed() -> int:
11         pass

```

```

12
13     def get_output_dir() -> str:
14         pass
15
16     def get_log_cfg_file_path() -> str:
17         pass
18
19     # --- Affichage ---
20     def print_config():
21         pass

```

Listing 55 – IConfig.py

S Annexe T – IEvent.py

```

1  # Interface
2  from abc import ABC, abstractmethod
3  import datetime
4  import string
5
6
7  class IEvent(ABC):
8
9      @abstractmethod
10     def get_event_time() -> datetime.datetime:
11         pass
12
13     @abstractmethod
14     def get_event_type() -> string:
15         pass
16
17     @abstractmethod
18     def set_event_time(evtString: datetime):
19         pass
20
21     @abstractmethod
22     def set_event_type(evtType: string):
23         pass
24
25     @abstractmethod
26     def get_event_id() -> int:
27         pass
28
29     @abstractmethod
30     def set_event_id(id: int):
31         pass
32
33     @abstractmethod
34     def print_event():
35         pass
36
37     '''
38     @abstractmethod
39     def foo(x: int, y: int):
40         pass
41     '''

```

Listing 56 – IEvent.py

T Annexe U – IGateway.py

```

1
2  # Interface
3  from abc import ABC, abstractmethod
4

```

```

5 from net.lectnam.rcp103.tp2 import IGateway
6 from net.lectnam.rcp103.tp2 import IQueue
7 from net.lectnam.rcp103.tp2 import IServer
8
9 class IGateway(ABC):
10
11     @abstractmethod
12     def is_empty(self) -> bool:
13         pass
14
15     @abstractmethod
16     def get_queue() -> IQueue:
17         pass
18
19     @abstractmethod
20     def set_queue(q: IQueue):
21         pass
22
23     @abstractmethod
24     def get_server() -> IServer:
25         pass
26
27     @abstractmethod
28     def set_server(server: IServer):
29         pass
30
31     '''
32     @abstractmethod
33     def foo(x: int, y: int):
34         pass
35     '''

```

Listing 57 – IGateway.py

U Annexe V – IMessage.py

```

1
2 # Interface
3 from abc import ABC, abstractmethod
4
5 from net.lectnam.rcp103.tp2.IServer import IServer
6
7 class IMessage(ABC):
8
9     @abstractmethod
10    def get_source():
11        pass
12
13    @abstractmethod
14    def get_destination():
15        pass
16
17    @abstractmethod
18    def get_message_id():
19        pass
20
21    @abstractmethod
22    def set_source(src: str):
23        pass
24
25    @abstractmethod
26    def set_destination(dst: IServer):
27        pass
28
29    @abstractmethod
30    def set_message_id(msg_id: int):
31        pass
32

```

```

33     @abstractmethod
34     def print_message():
35         pass
36
37     '''
38     @abstractmethod
39     def foo(x: int, y: int):
40         pass
41     '''

```

Listing 58 – IMessage.py

V Annexe W – IQueue.py

```

1
2 # Interface
3 from abc import ABC, abstractmethod
4
5 from net.lecnam.rcp103.tp2 import IMessage
6
7 class IQueue(ABC):
8
9     @abstractmethod
10    def is_empty(self) -> bool:
11        pass
12
13    @abstractmethod
14    def enqueue(msg: IMessage):
15        pass
16
17    @abstractmethod
18    def dequeue():
19        pass
20
21    @abstractmethod
22    def count_messages(self):
23        pass
24
25    @abstractmethod
26    def print_messages():
27        pass
28
29    '''
30    @abstractmethod
31    def foo(x: int, y: int):
32        pass
33    '''

```

Listing 59 – IQueue.py

W Annexe X – IScheduler.py

```

1
2 # Interface
3 from abc import ABC, abstractmethod
4 import datetime
5
6 from .IEvent import IEvent
7
8 class IScheduler(ABC):
9     @abstractmethod
10    def add_event(self, event: IEvent):
11        pass
12
13    @abstractmethod

```

```

14     def get_event(self):
15         pass
16
17     @abstractmethod
18     def get_current_time(self):
19         pass
20
21     @abstractmethod
22     def has_events(self):
23         pass
24
25     '''
26     @abstractmethod
27     def foo(x: int, y: int):
28         pass
29     '''

```

Listing 60 – IScheduler.py

X Annexe Y – IServer.py

```

1
2 # Interface
3 from abc import ABC, abstractmethod
4
5 from net.lecnam.rcp103.tp2 import IQueue
6 from net.lecnam.rcp103.tp2 import IMessage
7
8 class IServer(ABC):
9
10     @abstractmethod
11     def get_queue() -> IQueue:
12         pass
13
14     @abstractmethod
15     def set_queue(queue: IQueue):
16         pass
17
18     @abstractmethod
19     def get_server_id():
20         pass
21
22     @abstractmethod
23     def set_server_id(server_id: int):
24         pass
25
26     @abstractmethod
27     def print_server():
28         pass
29
30     @abstractmethod
31     def listen():
32         pass
33
34     '''
35     @abstractmethod
36     def foo(x: int, y: int):
37         pass
38     '''

```

Listing 61 – IServer.py

Y Annexe Z – logging_{conf}.cnf

```

1 [loggers]

```

```

2 keys=root
3
4 [handlers]
5 keys=consoleHandler,fileHandler
6
7 [formatters]
8 keys=simpleFormatter
9
10 [logger_root]
11 level=DEBUG
12 handlers=consoleHandler,fileHandler
13
14 [handler_consoleHandler]
15 class=StreamHandler
16 level=DEBUG
17 formatter=simpleFormatter
18 args=(sys.stdout,)
19
20 [handler_fileHandler]
21 class=FileHandler
22 level=DEBUG
23 formatter=simpleFormatter
24 args=('tp2.log', 'a')
25
26 [formatter_simpleFormatter]
27 format=%(asctime)s %(levelname)s %(name)s: %(message)s
28 datefmt=%Y-%m-%d %H:%M:%S

```

Listing 62 – logging_{conf}.cnf

Z Annexe AA – Message.py

```

1 from .IMessage import IMessage
2
3 class Message(IMessage):
4     """ Message envoy vers la passerelle """
5
6     message_id: int
7     source: str
8     destination: str
9     timestamp: float
10
11     def __init__(self, message_id, source, destination, timestamp=0.0):
12         self._message_id = message_id
13         self._source = source
14         self._destination = destination
15         self._timestamp = timestamp
16
17     ### getters du message ###
18     def get_message_id(self):
19         return self._message_id
20
21     def get_source(self):
22         return self._source
23
24     def get_destination(self):
25         return self._destination
26
27     def get_timestamp(self):
28         return self._timestamp
29
30     def set_message_id(self, message_id):
31         self._message_id = message_id
32
33     def set_source(self, source):
34         self._source = source
35
36     def set_destination(self, destination):

```



```

37         self._destination = destination
38
39     ### setters du message ###
40     def set_timestamp(self, temps):
41         self._timestamp = temps
42
43     # --- Affichage ---
44     def print_message(self):
45         message_str = (f"[Message] ID={self._message_id} | src={self._source} "
46                       f"| dst={self._destination} | timestamp={self._timestamp:.4f}")
47         print(message_str)
48         return message_str

```

Listing 63 – Message.py

Annexe AB – MessageImpl.py

```

1 from .IMessage import IMessage
2 from net.lecnam.rcp103.tp2.IServer import IServer
3
4 class MessageImpl(IMessage):
5     """ Message envoy vers la passerelle """
6
7     message_id: int
8     source: str
9     destination: IServer
10    timestamp: float
11
12    def __init__(self, message_id, source, destination, timestamp=0.0):
13        self._message_id = message_id
14        self._source = source
15        self._destination = destination
16        self._timestamp = timestamp
17
18    ### getters du message ###
19    def get_message_id(self):
20        return self._message_id
21
22    def get_source(self):
23        return self._source
24
25    def get_destination(self):
26        return self._destination
27
28    def get_timestamp(self):
29        return self._timestamp
30
31    def set_message_id(self, message_id):
32        self._message_id = message_id
33
34    def set_source(self, source):
35        self._source = source
36
37    def set_destination(self, destination):
38        self._destination = destination
39
40    ### setters du message ###
41    def set_timestamp(self, temps):
42        self._timestamp = temps
43
44    # --- Affichage ---
45    def print_message(self):
46        return (f"[Message] ID={self._message_id} | src={self._source} "
47              f"| dst={self._destination.print_server()} | timestamp={self._timestamp:.4f}")

```

Listing 64 – MessageImpl.py

Annexe AC – MessageImplTest.py

```
1 #!/usr/bin/python3
2
3 # Commande pour lancer le programme :
4 # PYTHONPATH=. python3 net/lecnam/rcp103/tp2/MessageImplTest.py
5 # en Linux/WSL: python3 -m net.lecnam.rcp103.tp2.MessageImplTest
6 # sous Windows/PowerShell: python MessageImplTest.py
7
8 from datetime import datetime
9 import traceback
10 import logging
11
12 from net.lecnam.rcp103.tp2.EventImpl import EventImpl
13 from net.lecnam.rcp103.tp2.IEvent import IEvent
14 from net.lecnam.rcp103.tp2.IMessage import IMessage
15 from net.lecnam.rcp103.tp2.MessageImpl import MessageImpl
16
17 try:
18     print("+++ START MessageImplTest fire")
19     impl = MessageImpl(1, "Alice", "Bob", 1.21)
20     src = impl.get_source()
21     dst = impl.get_destination()
22     print(f"Source: {src}, Destination: {dst}")
23     res = impl.print_message()
24     print("+++ Pretty print = " + str(res))
25     print("+++ END MessageImplTest ignite")
26
27 except RuntimeError as error:
28     print("Erreur au Runtime dans MessageImplTest")
29     print(f"A {type(error).__name__} has occurred.")
30     exit(42)
31 except Exception as exception :
32     print("Exception dans MessageImplTest/Main")
33     print(f"A {type(exception).__name__} has occurred.")
34     print("")
35     traceback.print_exc()
36     print("")
37     print(exception)
38     exit(42)
```

Listing 65 – MessageImplTest.py

Annexe AD – SchedulerImpl.py

```
1 # Interface
2 from abc import ABC, abstractmethod
3 import datetime
4
5 from net.lecnam.rcp103.tp2.IEvent import IEvent
6 from net.lecnam.rcp103.tp2.IScheduler import IScheduler
7
8 class SchedulerImpl(IScheduler):
9
10     events: list
11     current_time: float
12
13     def __init__(self):
14         self.events = []
15         self.current_time = 0.0
16
17     def add_event(self, event):
18         t = event.get_event_time()
19         for i, e in enumerate(self.events):
20             if (e.get_event_time() > t):
21                 self.events.insert(i, event)
22         return
```

```

23     self.events.append(event)
24
25     def get_event(self):
26         if not self.events:
27             return None
28         event = self.events.pop(0)
29         self.current_time = event.get_event_time()
30         return event
31
32     def get_current_time(self):
33         return self.current_time
34
35     def has_events(self):
36         return bool(self.events)
37
38     def print_scheduler(self):
39         resultat = ""
40         for event in self.events:
41             resultat += f"Event ID: {event.eventID}, Type: {event.eventType}, Time: {event.eventTime}\n"
42         return resultat

```

Listing 66 – SchedulerImpl.py